

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

ในบทนี้อธิบายถึงรายละเอียดความเป็นมาของเทคโนโลยีเว็บแต่เดิมก่อนที่จะพัฒนามาเป็นเทคโนโลยีเว็บเซิร์ฟวิส ความหมายของเว็บเซิร์ฟวิสและเทคโนโลยีอื่นๆ ที่เกี่ยวข้องกับเทคโนโลยีเว็บเซิร์ฟวิส

2.1. ความเป็นมาของการพัฒนาเทคโนโลยีเว็บเซิร์ฟวิส

เทคโนโลยีเว็บอาจแบ่งการประยุกต์และพัฒนาออกได้เป็น 3 ยุค ดังนี้

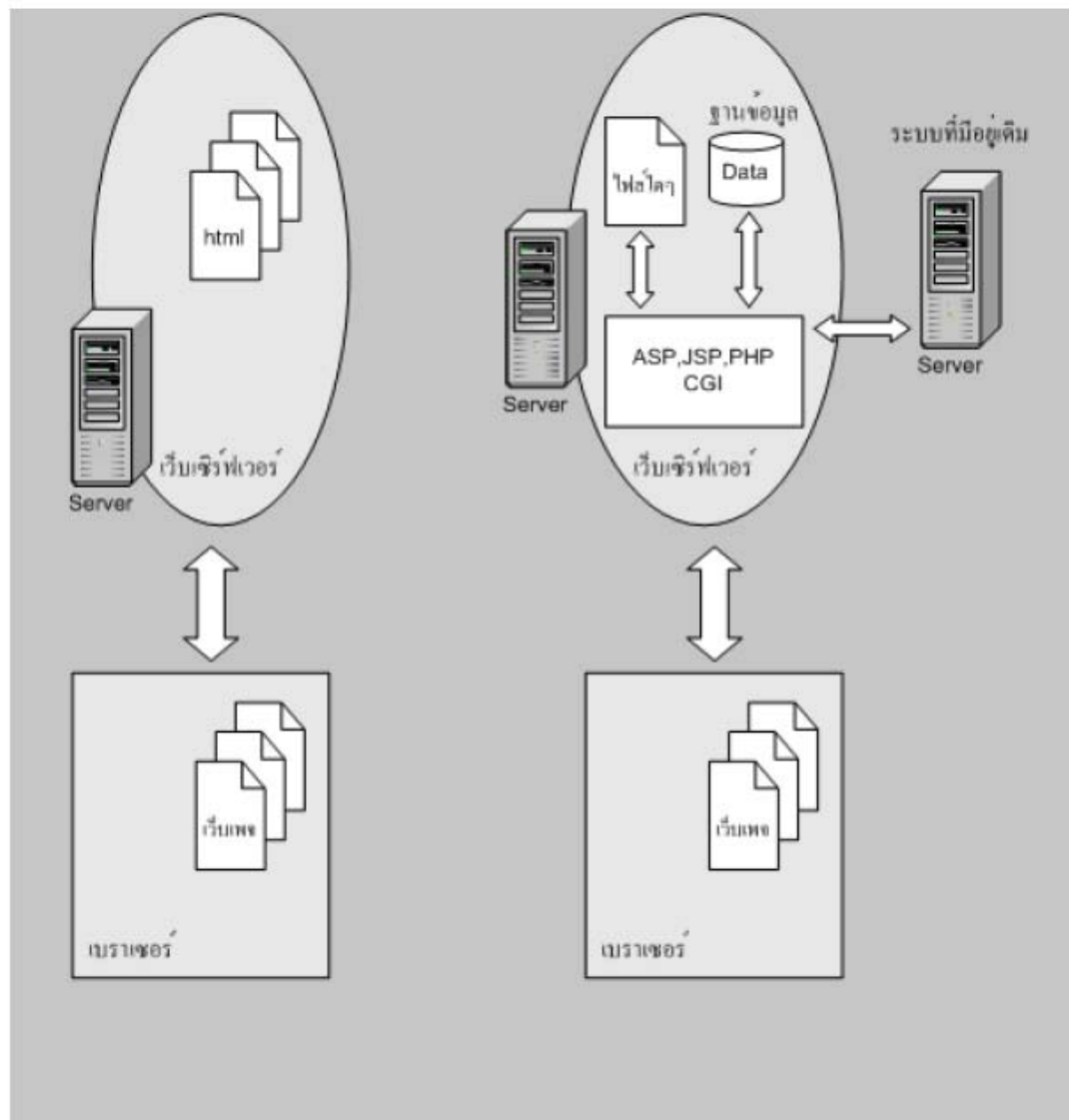
2.1.1. Static Web

ยุคเริ่มต้นของเว็บนั้นเป็นยุคที่มีการใช้บราวเซอร์เรียกเว็บเพจที่สร้างขึ้นมาจากภาษา HTML ทั้งหมดหรือมีสคริปต์ทางฝั่งไคลเอนต์ ตัวอย่างเช่น JavaScript, VBScript เป็นต้น ไว้เป็นไฟล์บนเซิร์ฟเวอร์ ผู้ใช้งานเรียกดูผลการประมวลผลผ่าน HTTP โปรโตคอล ในรูปแบบ HTML ซึ่งไฟล์เว็บเพจที่เป็นสคริปต์เหล่านี้ไม่สามารถติดต่อกับองค์ประกอบอื่นๆ ของทางฝั่งเซิร์ฟเวอร์ได้ เนื่องจากคุณลักษณะของเว็บเพจมีลักษณะคงที่ หรือมีการเปลี่ยนแปลงน้อยจึงเรียกว่า ยุคสแตติกเว็บ

2.1.2. Dynamic Web

เนื่องด้วยรูปแบบการแสดงผลและประมวลผลแบบสแตติกเว็บนั้นตายตัว จึงมีการพัฒนาให้เว็บมีความสามารถในการติดต่อกับองค์ประกอบอื่นๆ ทางฝั่งเซิร์ฟเวอร์ โดยทำการพัฒนาโปรแกรมทางฝั่งเซิร์ฟเวอร์ทำงานตามการเรียกขอบริการของไคลเอนต์ผ่านทางโปรโตคอล HTTP และให้มีการเชื่อมโยงกับโปรแกรมเฉพาะตามที่ทำการเขียนสคริปต์ขึ้นมา หรือเขียนสคริปต์ขึ้นเพื่อใช้ความสามารถในการประมวลผลของเซิร์ฟเวอร์เพื่อทำงานบางอย่าง เช่น สร้างห้องสนทนา(chat room) กระดานตอบ-ถาม(webboard) เป็นต้นโดยมีการใช้เทคโนโลยี CGI (Common Gateway Interface) ในการทำไดนามิกเว็บ CGI คือ โปรแกรมที่ทำงานอยู่บนฝั่งเซิร์ฟเวอร์ เมื่อผู้ใช้มีการเรียกใช้ CGI เมื่อใด CGI ก็จะทำงานตามหน้าที่ที่ถูกเขียนสคริปต์ขึ้นมา ซึ่งอาจมีการส่งผลลัพธ์ตอบกลับไปยังผู้ใช้ หรือไม่มีการส่งผลลัพธ์ไปยังผู้ใช้โดยขึ้นอยู่กับว่ามีการระบุหน้าที่ไว้ในสคริปต์ CGI ไว้เป็นอย่างไร ต่อมาจึงมีการพัฒนาเทคโนโลยีหลายอย่างที่มีหลักการคล้ายกับ CGI เพื่อทำงานทางฝั่งเซิร์ฟเวอร์ เช่น ASP(Active server Pages), JSP(JavaServer Pages), PHP การพัฒนาโดยใช้เทคโนโลยี CGI เรียกว่ายุคไดนามิกเว็บ

ดังนั้นความแตกต่างระหว่างสแตติกเว็บกับไดนามิกเว็บจะแตกต่างกันทั้งรูปแบบการทำงานและการประมวลผล



รูป 2.1 เปรียบเทียบลักษณะการทำงานระหว่างเทคโนโลยี Static Web และ Dynamic Web

2.2. หลักพื้นฐานของเว็บเซอร์วิส

2.2.1. ความหมายของเว็บเซอร์วิส

เว็บเซอร์วิสเป็นระบบซอฟต์แวร์ที่ออกแบบมาเพื่อสนับสนุนการทำงานระหว่างคอมพิวเตอร์กับคอมพิวเตอร์ผ่านระบบเครือข่าย โดยที่ภาษาที่ใช้ในการติดต่อสื่อสารระหว่างคอมพิวเตอร์คือภาษา XML)

การอธิบายวิธีการใช้งานของเว็บเซอร์วิสนั้นจะอธิบายโดยใช้ภาษาวิสเดิล WSDL (Web Services Description Language) ซึ่งเป็นภาษา XML ประเภทหนึ่ง

ระบบอื่นๆ จะสามารถติดต่อและทำงานกับเว็บเซอร์วิส โดยใช้ โปรโตคอลที่ชื่อว่า SOAP (Simple Object Protocol) ซึ่งใช้ภาษา XML เป็นมาตรฐานในการติดต่อระหว่างระบบโดยผ่านทางโปรโตคอลอื่นที่ใช้ในการส่งข้อมูลบนเว็บ อาทิเช่น โปรโตคอล HTTP

2.2.2. เว็บเซอร์วิสกับเว็บแอปพลิเคชัน

เทคโนโลยีในการกระจายข่าวสารข้อมูลทางอินเทอร์เน็ตในปัจจุบันก็คือ เว็บเพจ แต่จากการที่มันมีความสามารถที่จะทำงานได้ด้วยการรวมภาษาทั้งไคลเอนท์และเซิร์ฟเวอร์ไสค์สคริปต์ ไว้ในตัวเองเช่นภาษา VBScript, Java Script หรือ ASP, PHP, JSP นั้นทำให้เว็บเพจมีลักษณะคล้ายแอปพลิเคชันจึงถูกเรียกรวมกันว่าเว็บแอปพลิเคชัน

เว็บแอปพลิเคชัน นั้นสามารถตอบสนองความคิดdistributed processing (Distributed Processing) ได้ในระดับหนึ่งซึ่งก็คือ การแบ่งการประมวลผลไว้ที่ฝั่งไคลเอนท์และฝั่งเซิร์ฟเวอร์และมักจะมีการใช้ดาต้าเบสควบคู่กับการทำเว็บแอปพลิเคชันไปด้วยตามความต้องการในการทำE-Business และ E-Commerce ที่กำลังเป็นที่นิยมในปัจจุบัน และเกิดปัญหาที่ตามมาคือ เรื่องของการจ่ายเงินหรือที่เรียกว่า E-payment หรือ Payment-Gateway ซึ่งเว็บแอปพลิเคชันที่ทำ E-Commerce ต้องใช้บริการจากธนาคารออนไลน์ในการจัดเก็บเงินกับลูกค้า เพราะด้วยเทคโนโลยีนี้การให้บริการเก็บเงินจากธนาคารออนไลน์จำเป็นที่ผู้ค้าต้องไปทำการตกลงกับธนาคารและเขียนโปรแกรมให้ตรงตามมาตรฐานที่ธนาคารออนไลน์กำหนดไว้

ด้วยปัญหายุ่งยากในการค้นหา ติดต่อและตกลงในการขอใช้บริการเก็บเงินจากธนาคารออนไลน์แนวคิดเว็บเซอร์วิสจึงดูเหมือนเป็นทางออกของปัญหานี้ ความเด่นของเทคโนโลยีเว็บเซอร์วิสก็คือ การทำระหว่างเว็บกับเว็บสามารถติดต่อสื่อสารกันได้ด้วยเอกสาร XML ที่ทั้งคนและคอมพิวเตอร์เข้าใจ และคอมพิวเตอร์ยังสามารถนำข้อมูลนั้นไปประมวลผลต่อได้ด้วยเอกสาร XML นี้เองทำให้เว็บสามารถส่งข้อมูลที่จำเป็นไปให้อีกเว็บหนึ่งทำงานบางอย่างให้หรือใช้บริการนั่นเอง ทำให้เป็นการง่ายที่จะเขียนโปรแกรมที่จะติดต่อสื่อสารหรือขอใช้บริการเก็บเงินจากธนาคารออนไลน์แต่สำหรับเว็บแอปพลิเคชันที่ใช้การส่งข้อมูลเป็น HTML ทำให้ข้อมูลนั้นไม่สามารถนำไปใช้ได้ การเขียนโปรแกรมจึงยุ่งยากตามที่กล่าวด้านบน

แนวคิดของเว็บเซอร์วิส ก็คือ เว็บที่สามารถทำงานอะไรบางอย่างหรือก็คือให้บริการบางอย่างจากการร้องขอจากต่างเซิร์ฟเวอร์ ด้วยเหตุนี้ทำให้เทคโนโลยีเว็บเซอร์วิสถือได้ว่า Distributed processing มากกว่าเว็บแอปพลิเคชัน และเมื่อประกอบกับการที่เว็บเซอร์วิส มี UDDI ทำให้เว็บเซอร์วิสสามารถค้นหาบริการต่างๆ ที่ต้องการได้จากทั่วทุกมุมโลก

2.2.3. ประโยชน์ของการทำงานร่วมกันระหว่างแอปพลิเคชันกับเว็บเซอร์วิส

เว็บเซอร์วิสช่วยให้การเข้าถึงข้อมูลสารสนเทศจากแอปพลิเคชันที่ต่างกันเป็นไปได้โดยง่าย โดยแอปพลิเคชันนั้นๆ สามารถเขียนด้วย Java และรันอยู่บน Sun Solaris Application Server หรืออาจจะเขียนด้วย C++ และรันอยู่บนวินโดวส์เอ็นที(Windows NT) หรืออาจจะเขียนด้วยเพิร์ล (Perl) และรันอยู่บนเครื่องลินุกซ์(Linux) ซึ่งมาตรฐานของเว็บเซอร์วิสทำให้อินเตอร์เฟซของ แอปพลิเคชันเหล่านี้ ถูกอธิบายโดย WSDL และทำให้อยู่ในมาตรฐานของ UDDI หลังจากนั้น จึงสามารถติดต่อสื่อสารถึงกันโดย XML ผ่าน SOAP อินเทอร์เน็ต

เว็บเซอร์วิสสามารถถูกเรียกใช้ภายในองค์กรเองหรือจากภายนอกองค์กร โดยผ่านไฟร์วอลล์ ดังนั้นจึงมีองค์กรใหญ่ๆ มากมาย กำลังพัฒนาระบบที่มีอยู่ของตน ให้เข้ากับเว็บเซอร์วิสซึ่ง นับเป็นการลงทุนที่คุ้มค่า เนื่องจากเว็บเซอร์วิสสามารถเพิ่มศักยภาพในการทำงานขององค์กร อีกทั้งลดค่าใช้จ่ายในการจัดการทรัพยากรขององค์กรได้อีกทางหนึ่ง

นอกจากนั้นเว็บเซอร์วิสยังสามารถใช้ร่วมกับเว็บแอปพลิเคชันโดยส่งผ่านข้อมูลทาง อินเทอร์เน็ตได้อีกด้วยซึ่งนับเป็นวิธีที่มีประสิทธิภาพในการติดต่อสื่อสารกับลูกค้าหรือหุ้นส่วน ถึงแม้จะต้องคำนึงถึงระบบรักษาความปลอดภัย และการจัดการรายการของข้อมูลอยู่ก็ตาม แต่ เว็บเซอร์วิสได้ใช้มาตรฐานทั่วไปของอินเทอร์เน็ตเรื่องดังกล่าวจึงนับเป็นเรื่องธรรมดาของการ สื่อสารผ่านระบบอิเล็กทรอนิกส์

2.2.4. การประยุกต์ใช้งาน

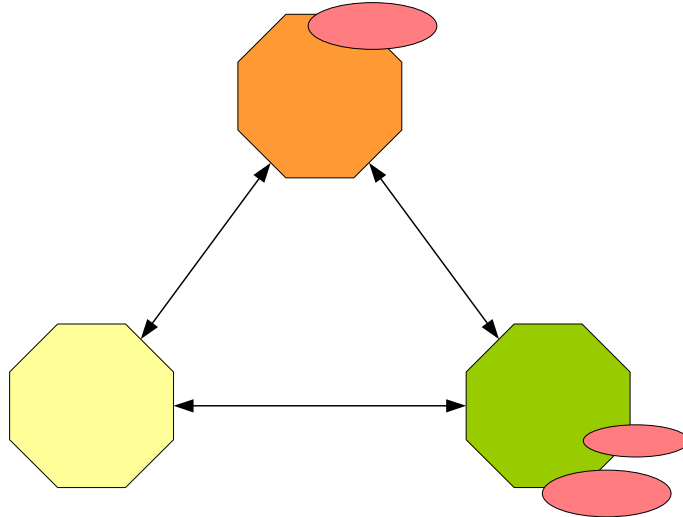
ขณะเดียวกัน เว็บเซอร์วิสเป็นพื้นฐานสำคัญของการบริการใหม่ๆ ในลักษณะที่ต้องการ เชื่อมโยง ระบบสารสนเทศที่มีความแตกต่างเข้าด้วยกันอย่างอัตโนมัติ เช่น การพาณิชย์ อิเล็กทรอนิกส์ระหว่างธุรกิจกับธุรกิจ (B2B), ระบบการบริการรัฐบาลอิเล็กทรอนิกส์ ใน ลักษณะที่มีการเชื่อมบริการของภาครัฐกับภาครัฐเข้าด้วยกัน เพื่อทำให้เกิดการบริการต่อ ประชาชนจากจุดเดียวได้ (One Stop Services) เป็นต้น

ตัวอย่างพื้นฐานหนึ่ง คือ การเชื่อมโยงระหว่างระบบการสั่งซื้อ ของบริษัทผู้ขายบริษัทหนึ่ง ไปยังระบบสารสนเทศการ จัดส่งของบริษัทจัดส่งอีกบริษัทหนึ่ง จะเป็นไปได้ง่ายมากขึ้นด้วย เทคโนโลยีของเว็บเซอร์วิส พื้นฐานดังกล่าวนี้ จะทำให้ "การจัดการห่วงโซ่อุปทานด้วยสื่อ อิเล็กทรอนิกส์" (e-SCM:Supply Chain Management) และธุรกิจผ่านสื่ออิเล็กทรอนิกส์ที่มี ประสิทธิภาพ และมีความคุ้มค่ามากขึ้น โดยมีการแลกเปลี่ยนข้อมูลธุรกิจ ระหว่างองค์กรใน ห่วงโซ่อุปทาน

ทั้งนี้แต่ละบริษัทจะมีระบบงาน (แอปพลิเคชัน) และใช้รูปแบบของข้อมูล (ฟอร์แมต มาตรฐาน) ที่แตกต่างกันได้ ซึ่งเมื่อมีการส่งข้อมูลจากองค์กรหนึ่งไปอี้องค์กรหนึ่ง ระบบ คอมพิวเตอร์ของบริษัทผู้รับ สามารถนำข้อมูล ที่ได้ไปประมวลผลต่อได้ทันที การเชื่อมโยง

ระบบอีคอมเมิร์ซของหน้าร้าน กับระบบหลังร้าน แล้วยังเชื่อมโยง กับระบบสารสนเทศของพันธมิตรทางการค้านี้ จะทำให้การทำธุรกิจผ่านสื่ออิเล็กทรอนิกส์ มีความคุ้มค่า มีประสิทธิภาพ และสร้างผลกำไรให้กับบริษัทได้ นั่นเอง

2.2.5. Web Services Model



รูปที่ 2.1 แสดง Web Services Model

Requestor	เป็นใครก็ตามที่ต้องการเรียกใช้บริการจาก Provider ซึ่งสามารถค้นหาบริการที่ต้องการได้จาก UDDI registry หรือ Service Registry หรือติดต่อจาก Provider โดยตรง
Registry	ทำหน้าที่เป็นตัวกลางให้ Provider มาลงทะเบียนไว้ โดยใช้ WSDL ไฟล์ บอกรายละเอียดของบริษัทและบริการที่มีให้ ซึ่งอาจจะใช้หรือไม่ใช้ก็ได้
Provider	เป็นผู้ให้บริการ มีหน้าที่ในการเปิดบริการเพื่อรองรับการขอใช้บริการจาก Requestor ที่เรียกเข้ามาขอใช้

ตาราง 2.1 ตารางแสดงโครงสร้างของ Web Service Model

2.2.6. XML (The Extensible Markup Language 1.0)

XML (The Extensible Markup Language 1.0) เป็นภาษา Markup ที่เป็นเท็กซ์เบส(text-based) ซึ่งทำให้เป็น มาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตอย่างรวดเร็ว ผู้ที่ทำหน้าที่รับผิดชอบ และกำหนดมาตรฐานของ XML คือ World Wide Web Consortium (W3C) ความแตกต่างระหว่าง XML กับ HTML คือ HTML ถูกนำมาใช้ในการสร้าง เว็บเพจ ที่สามารถแสดงผลได้โดยโปรแกรมบราวเซอร์ แต่ XML จะใส่แท็ก(tags) ได้อย่างอิสระ แล้วทำการส่ง XML ชุดนี้ไป ประมวลผลยังแอปพลิเคชันใด ๆ ที่สามารถใช้ข้อมูลใน XML นี้

XML & HTML

HTML ภาษาที่ใช้ในการเขียนเว็บมากที่สุดนั้นเป็นเพราะมีรูปแบบที่ง่ายต่อการแสดงผลของบราวเซอร์เนื่องจาก มีแท็กตายตัวที่สามารถบอกได้ว่าเมื่อเจอแท็กนี้จะแสดงผลอย่างไร เช่น เมื่อเจอแท็ก... ในเอกสารก็ให้แสดงข้อความที่อยู่ระหว่างแท็กเป็นตัวหนา แต่จะสังเกตได้ว่าคอมพิวเตอร์จะไม่เข้าใจว่าข้อความนั้นคืออะไร เพียงแต่รู้ว่าจะแสดงผลอย่างไร นั่นแสดงว่าไม่สามารถนำข้อมูลภายใน tag เหล่านี้ไปทำการประมวลใดๆ ได้เลย

XML เป็นภาษาที่มีลักษณะเป็น tag คล้าย HTML แต่ไม่ได้มุ่งที่การแสดงผล XML มุ่งที่การสื่อความหมายโดยอนุญาตให้ผู้ใช้งานสามารถกำหนดแท็กขึ้นได้เองเพื่อให้สื่อความหมายทางภาษาของมนุษย์ แต่คอมพิวเตอร์เองก็เข้าใจเช่นกัน ทำให้ข้อมูลระหว่างแท็กสามารถนำไปประมวลผลต่อได้ เช่น

```
<ComputerBook>
  <book>
    <name>เว็บเซอร์วิส</name>
    <price>10.00$</price>
  </book>
  <book>
    <name>xml</name>
    <price>10.00$</price>
  </book>
</ComputerBook>
```

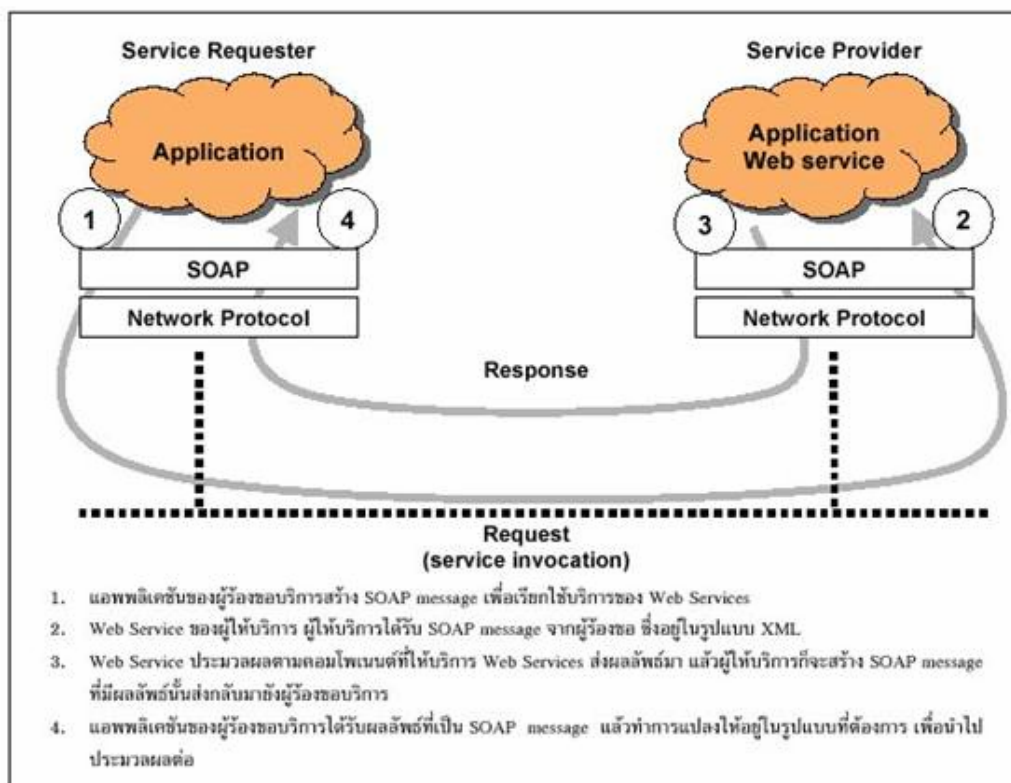
จากตัวอย่าง จะเห็นได้ว่ารูปแบบเอกสาร XML ไม่ได้บอกวิธีแสดงผลไว้ แต่เอกสารสามารถสื่อความหมายให้คอมพิวเตอร์เข้าใจได้ และนำค่าไปประมวลผลต่อได้ แต่ถ้าเราต้องการจะแสดงผลเราต้องใช้ควบคู่กันกับ HTML

2.2.7. SOAP(Simple Object Access Protocol)

SOAP กลายเป็นสิ่งที่มีความสำคัญสำหรับเว็บเซอร์วิสอย่างรวดเร็ว เป็นโปรโตคอลที่ผู้จัดทำเว็บเซอร์วิสไว้เลือกใช้ที่จะส่งเมสเสจระหว่างเว็บเซอร์วิส ซึ่ง SOAP เป็นทรานสปอร์ตโปรโตคอลที่มี XML เป็นพื้นฐานและใช้ HTTP เป็นโปรโตคอลร่วมในการส่งผ่านเครือข่าย SOAP จะระบุวิธีการเข้ารหัสส่วนหัว (Header Encoding) ของทั้ง HTTP และไฟล์ XML ให้อย่างชัดเจนทั้งใน ส่วนของการติดต่อไปยังคอมพิวเตอร์เครื่องหนึ่งและส่งผ่านข้อมูลไปให้รวมถึงระบุวิธีที่โปรแกรมซึ่งถูกเรียกนั้นจะส่งค่าคืนกลับมาด้วย

SOAP (Simple Object Access Protocol) เป็น XML-based โปรโตคอล (lightweight protocol) และใช้ HTTP เป็นโปรโตคอลร่วม สำหรับ การแลกเปลี่ยนข้อมูลในสภาวะแวดล้อมแบบกระจายศูนย์ (decentralized, distributed environment) SOAP ได้ กำหนดเมสเสจจิงโปรโตคอล (Messaging Protocol) ระหว่างผู้ขอบริการ (requestor) กับผู้ให้บริการ (provider)

เช่น ผู้ขอบริการสามารถติดต่อแลกเปลี่ยนข้อมูลกับผู้ให้บริการโดยใช้ RMI (Remote Method Invocation) ตามวิธีการของ โปรแกรมแบบออปเจ็ค บริษัทไมโครซอฟท์, ไอบีเอ็ม, โลตัส, UserLand และ DeveloperMenter ได้ร่วมกันกำหนดมาตรฐานของ SOAP ขึ้น ซึ่งต่อมาได้มีบริษัทอีก 30 กว่าบริษัทเข้าร่วมและ จัดตั้งเป็น W3C XML Protocol Workgroup ขึ้น SOAP ได้กำหนดรูปแบบพื้นฐานของการสื่อสารแบบกระจายขึ้นโดย การพัฒนา SOA แม้ว่า SOA จะไม่ได้กำหนด Messaging Protocol ไว้ แต่ SOAP ได้ถูกกำหนด ให้เป็น Services-Oriented Architecture Protocol เรียบร้อยแล้ว เนื่องจากมันได้ถูกใช้ในการพัฒนา SOA อย่างแพร่หลายแล้วนั่นเอง จุดเด่นของ SOAP ก็คือเป็นโปรโตคอลที่เป็นกลาง กล่าวคือ ไม่มีใครเป็นเจ้าของ และเป็นโปรโตคอล ที่ทำงานกับโปรโตคอลอื่นหลายชนิด การพัฒนาก่อนุญาตให้ทำได้อย่างอิสระตามแพลตฟอร์มระบบปฏิบัติการ แบบจำลองทางวัตถุ (Object model) และภาษาโปรแกรมของผู้ที่ทำการพัฒนา



รูป 2.2 แสดง การส่ง XML โดยใช้ SOAP

เนื่องจากจุดประสงค์หลักของการใช้งานเว็บเซอร์วิส เราต้องการให้แอปพลิเคชัน มีการทำงานกับแอปพลิเคชันที่ทำงานอยู่ในเครื่องอื่นโดยผ่านทางเครือข่าย ซึ่งเทคโนโลยีที่มีอยู่ในปัจจุบันที่ใช้มีการสื่อสารระหว่าง Object ในระยะไกล (Remote Procedure Calls : RPC) เช่น DCOM,EJB หรือ CORBA นั้นไม่ได้ถูกออกแบบมาใช้สำหรับโปรโตคอล HTTP (เครือข่ายที่

ปัจจุบันใช้งานแพร่หลาย และสะดวกมากที่สุดได้แก่เครือข่าย Internet ซึ่งให้การสื่อสารผ่านโปรโตคอล HTTP เป็นส่วนใหญ่)

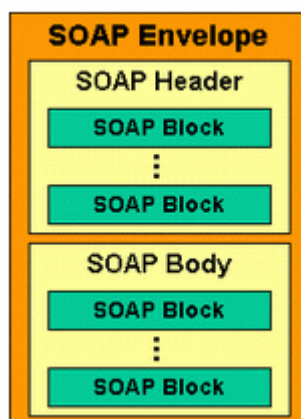
เทคนิค RPC ของเทคโนโลยีที่กล่าวข้างต้นนั้นต่างก็มีปัญหาในด้านการนำมาใช้งานในแง่ของความเข้ากันได้ของการเรียกใช้งานข้ามเทคโนโลยี เนื่องจากเป็นเทคโนโลยีเฉพาะของแต่ละค่าย ยกเว้น CORBA ผู้พัฒนาระบบจะต้องพัฒนาโปรแกรมที่มีความซับซ้อน และยังมีปัญหาในส่วน of Firewalls และ Proxy Server ด้วยเนื่องจากโดยปกติ เซอร์เวอร์จะปิดการสื่อสารที่ไม่ใช่โปรโตคอล HTTP ออกไป เพื่อความปลอดภัยของระบบที่มีการติดต่อสื่อสารกับภายนอก

ดังนั้นทางเลือกของการสื่อสารที่จะนำมาใช้ในการให้บริการเว็บเซอร์วิส ก็คือให้ทำงานอยู่บน โปรโตคอล HTTP ไปเลย ซึ่ง SOAP นอกจากจะทำงานบนโปรโตคอล HTTP แล้วยังเป็นมาตรฐานเปิดที่จะทำให้สามารถติดต่อสื่อสารกับเครื่องคอมพิวเตอร์ที่มีความแตกต่างกันทั้งระบบปฏิบัติการ, เทคโนโลยี รวมไปถึงภาษาที่ใช้ในการพัฒนาด้วยก็ได้

โครงสร้างของ SOAP

เอกสาร SOAP นั้นมีโครงสร้างในรูปแบบ XML ซึ่งเราสามารถแบ่งเป็นส่วนของเอกสารได้เป็น 3 ส่วนหลักดังนี้คือ

1. SOAP envelop เนื้อหาสาระ (Content) ของเอกสารทั้งหมด
2. SOAP header ส่วนเพิ่มเติมของเอกสาร SOAP ซึ่งจะมีก็ได้ หรือ ไม่มีก็ได้
3. SOAP body ส่วนที่ใช้ในการเรียกใช้งานเซอร์วิส และผลลัพธ์ที่ได้จากเซอร์วิส



รูป 2.3 โครงสร้างของเอกสาร SOAP

ตัวอย่างเอกสาร SOAP อย่างง่ายของการสอบถามราคา ดอกกุหลาบ จำนวน 100 ดอก

```

<soap:Envelope>
  <soap:Body>
    <GetPrice>
      <Item>Rose</Item>
      <Quantity>100</Quantity>
    </GetPrice>
  </soap:Body>
</soap:Envelope>

```

```

</soap:Body>
</soap:Envelope>

```

เรามาดูอีกตัวอย่างหนึ่งของเอกสาร SOAP ที่จะแสดงในรูปแบบด้านล่างเพื่อจะได้ทำความเข้าใจในส่วนต่างๆ ที่มีความซับซ้อนขึ้น แต่ก็อย่าจนเกินไปที่จะทำความเข้าใจได้ดังรูปที่ 2 SOAP Envelope ต่อไปนี้

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Header>
    <!-- ข้อมูลในส่วนของ Header -->
    <i:local xmlns:i="http://www.i3t.or.th/ws/">
      <i:currency>Bath</i:currency>
    </i:local>
  </soap:Header>
  <soap:Body>
    <!-- ข้อมูลในส่วนของ Body -->
    <GetPrice>
      <Item>Rose</Item>
      <Quantity>100</Quantity>
    </GetPrice>
  </soap:Body>
  <soap:Fault>
    <!-- ข้อมูลของ SOAP ในกรณีมีข้อผิดพลาด จาก SOAP Node -->
  </soap:Fault>
</soap:Envelope>

```

Element	Definition
<Envelope>	จะเป็น Root ของเอกสาร XML SOAP เสมอ โดยจากตัวอย่างจะใช้ Namespaces จาก "http://schemas.xmlsoap.org/soap/envelope/" (Schema for the SOAP/1.1 envelope) ดังนั้น Element ต่างๆ ในตัวอย่างนี้ที่อยู่ในเอกสาร XML ส่วนที่เป็นมาตรฐานของ SOAP เราจะมี Namespaces ที่เรากำหนดเป็น soap (<soap:Envelope>, <soap:Header>, <soap:Body>, <soap:Fault>) Attribute ใน Envelope Attribute encodingStyle ในตัวอย่างมีการอ้างการ encoding จาก "http://schemas.xmlsoap.org/soap/encoding/" (Schema for the SOAP/1.1 encoding) จะใช้ในกรณีที่เรามีการใช้ Parameter หลายชนิดข้อมูลในเอกสาร (Data Type) เช่น Boolean, String, Integer เป็นต้น
<Header>	จากตัวอย่างมีการเพิ่มเนื้อหาของเอกสาร SOAP เข้าไปในส่วนของ Header ในที่นี้คือ

	สกุลเงิน (Currency) ซึ่งส่วนนี้จะใช้ในส่วนของ Application ไม่ได้เป็นส่วนของมาตรฐานของ SOAP แต่ผู้ใช้งานเป็นผู้กำหนดใช้เอง (User-defined) Attribute ใน Header ใน <Header> Element อาจจะมีการใส่ Attribute mustUnderstand เพื่อให้ฝั่งที่รับเอกสาร SOAP ให้ทำการประมวลผลในส่วน Header ด้วยโดยกำหนดค่าเป็น Boolean เช่น <pre><i:local xmlns:i="http://www.i3t.or.th/ws/"> <i:country mustUnderstand="1">Thailand</i:country> <i:currency mustUnderstand="0">Bath</i:currency> </i:local></pre> ค่าของ mustUnderstand ถ้าเป็น "0" แสดงว่าไม่ต้องประมวลผล ถ้าเป็น "1" : true คือจำเป็นต้องมีการประมวลผล (โดยค่า default = "0" : fault)
<Body>	โดยปกติแล้วเอกสาร SOAP จะต้องมีส่วนของ Body เพราะเป็นส่วนเป็นเนื้อหาสาระจริงๆ ของ SOAP จากตัวอย่างเราต้องการสอบถามราคาของดอกกุหลาบจำนวน 100 ดอก ซึ่ง <GetPrice>, <Item> และ <Quantity>เป็น Element ที่ใช้งานใน Application ไม่ใช่มาตรฐานของ SOAP
<Fault>	เป็นส่วนที่จะถูกใช้เมื่อมีข้อผิดพลาดในการประมวลผลของเอกสาร SOAP ซึ่งโดยปกติจะเห็นเฉพาะที่เป็นเอกสารตอบกลับเท่านั้น (Reply Message หรือ Answer Message) ซึ่งตัวอย่างจะเป็นเอกสารที่เป็นการร้องขอบริการ (Request Message) ซึ่งไม่น่าจะมีส่วนของ <Fault> Element แต่ที่เขียนไว้เพื่อให้เห็นภาพรวมของโครงสร้างเอกสาร SOAP ทั้งหมดก่อน ตัวอย่างข้อมูลใน <Fault> Element ที่มีการเกิดข้อผิดพลาด

ตาราง 2.2 ตารางแสดงโครงสร้างของ SOAP

<pre><soap:Fault> <faultcode>Client</faultcode> <faultstring>Invalid Request</faultstring> </soap:Fault></pre>
--

ตารางอธิบาย Fault Element

Sub Element	คำอธิบาย
<faultcode>	ชื่อของการเกิดข้อผิดพลาด (ดูตารางด้านล่าง)
<faultstring>	คำอธิบายของข้อผิดพลาด
<faultactor>	ชื่อของแหล่งที่เกิดข้อผิดพลาด
<detail>	ข้อมูลรายละเอียดเพิ่มเติมของข้อผิดพลาดนั้น

ตาราง 2.3 ตารางอธิบาย Element ของ SOAP

ตารางอธิบาย faultcode Sub Element

Error (faultcode)	คำอธิบาย
VersionMismatch	มีข้อผิดพลาดของการอ้าง Namespace ของ Element
MustUnderstand	ในการประมวลผลของ Sub Element ใน Header ที่มีการกำหนด attribute mustUnderstand เป็น "1" ไม่สามารถประมวลผลได้ (not understood)
Client	เอกสาร SOAP มีข้อผิดพลาดอันเนื่องมาจากมีโครงสร้างทางผิด หรือมีข้อมูลที่ไม่ถูกต้อง
Server	มีข้อผิดพลาดจาก Server ไม่สามารถประมวลผลเอกสารได้

ตาราง 2.4 ตารางอธิบาย Sub Element ของ SOAP

ท่านจะเห็นว่า SOAP Envelope ก็เป็นเอกสารที่อยู่ในรูปแบบ XML ทั่วไป แต่สิ่งที่ทำให้ SOAP มีความสามารถมากขึ้นก็คือ SOAP ได้มีการกำหนดโครงสร้างของเอกสารเป็นส่วนๆ โดยอธิบายว่าส่วนใดมีหน้าที่อะไร (What) มีข้อมูลอะไรอยู่ในส่วนนั้น และใคร (Who) คือผู้ที่จะต้องสนใจในส่วนนั้น ๆ และกล่าวถึงว่าส่วนใดจำเป็นที่ต้องมีในเอกสาร และส่วนใดเป็นส่วนที่เพิ่มเติมอาจจะมีหรือไม่มีส่วนนั้นก็ได้ขึ้นอยู่กับความจำเป็นในการใช้งาน ทำให้การพัฒนาโปรแกรมมีทิศทางที่ค่อนข้างชัดเจน แต่สามารถเพิ่มขยายได้ในอนาคตอีกด้วย

2.2.8. SOAP Intermediaries

จากตัวอย่างเอกสาร SOAP นั้นจะเห็นได้ว่าในส่วนของ <Header> Element จะเป็นส่วนที่เราจะขยายสารสนเทศ (information) ใหม่ๆ เข้าไปใน SOAP ได้ซึ่งตัวอย่างเรามีการอธิบายให้ผู้รับเอกสารตัวอย่างนี้ให้ทราบว่าในการสอบถามราคานี้ให้ใช้เงินในสกุลเงินบาท เป็นต้น ซึ่งเราจะเรียกการขยายในส่วนนี้ว่าเป็นการขยายในแนวดิ่ง (Vertical extensibility) แต่ถ้าเราจะมาดูในด้านการขยายในแนวราบ (Horizontal extensibility) ของ SOAP นั้นก็คือทำอย่างไรที่จะให้มี

การส่งเอกสาร SOAP ไปยังปลายทางโดยเส้นทางของการเดินทางของเอกสารอาจไม่เป็นลักษณะแบบจุดต่อจุด (Point-to-point) แต่อาจจะเป็นการส่งเอกสารตามเส้นทางโดยเอกสารยังคงเป็นเอกสารตัวเดิมดังรูป

ความจำเป็นในการใช้ SOAP Intermediary

SOAP Intermediaries จะทำหน้าที่เป็นตัวผ่าน หรือตัวกลางในการส่งเอกสาร SOAP จากต้นทาง ในที่นี้คือ Requestor ไปยัง Provider เพื่อสร้างกระบวนการบางอย่างกับเอกสาร ซึ่งในการสร้างเส้นทางในการส่งแบบนี้อาจมีจุดประสงค์ดังต่อไปนี้คือ

1. Intermediary ทำหน้าที่เป็น Crossing trust domains เนื่องจากการทำงานของ Web Services เป็นการเรียกใช้งานแบบระยะไกล หรือมีการทำงานแบบระบบกระจาย (Distributed System) ดังนั้นวิธีการในด้านความปลอดภัยจึงเป็นส่วนหนึ่งที่ต้องคำนึงถึง ซึ่งวิธีการหนึ่งก็คือให้มีการส่งเอกสารมายังจุดหนึ่งที่มีความน่าเชื่อถือได้ก่อน แล้วส่งต่อไปยังปลายทางอีกทีหนึ่ง ซึ่งหลักการใช้ Trust Domains เราอาจจะดูตัวอย่างง่ายๆ เช่น ในองค์กรเราอาจจะมีเครือข่าย Intranet อยู่ภายในองค์กร เครื่องคอมพิวเตอร์แต่ละเครื่องอาจจะสามารถติดต่อไปยัง Server ได้โดยตรง แต่ถ้าต้องการออกไปยัง Internet หรือ Extranet ที่อยู่นอกองค์กรจะต้องผ่าน Trust Domains ก่อน ในที่นี้อาจจะเป็น Firewalls หรือ Virtual Private Network (VPN) gateways เพื่อความปลอดภัย

2. Intermediary ทำหน้าที่เป็นตัวขยายการทำงานแบบระบบกระจาย (Distributed System) ถ้าเรามองดูเฉพาะแง่ของการขอใช้บริการจาก Requestor ไปยังผู้ให้บริการ Provider โดยผู้ให้บริการจะมีการกระบวนการทำงานบางอย่างภายในและตอบกลับไปยังผู้ขอใช้ ซึ่งการทำงานแบบนี้ก็อยู่บนพื้นฐานของ Request and Respond นั่นเอง แต่ถ้าระบบของเรามีการขยายใหญ่ขึ้นสิ่งที่เราจะต้องดูเพิ่มขึ้นก็คือความสามารถในการรองรับงานของผู้ให้บริการด้วย ดังนั้นก่อนที่จะมีการส่งเอกสาร SOAP ไปยังเซิร์ฟเวอร์ ของผู้ให้บริการนั้นอาจจะมีการผ่าน Intermediary Server ก่อนเพื่อทำหน้าที่เป็น Buffer รับเอกสาร SOAP มาพักไว้และส่งเอกสารให้ผู้ให้บริการ (Provider) อีกทอดหนึ่ง การทำงานเช่นนี้จะช่วยให้ Server ของ ผู้ให้บริการไม่รับภาระงานมากเกินไปในช่วงเวลาใดเวลาหนึ่งจนอาจทำให้เกิดความเสียหายได้ และ Intermediary อาจจะมีการรวบรวมเอกสารแล้วส่งมาให้ผู้ให้บริการเป็นลักษณะ Batch เพื่อเพิ่ม Performance การใช้งาน Intermediary ลักษณะนี้ผู้ขอใช้บริการอาจจะไม่เห็น Intermediary แต่จะมองเห็นว่าติดต่ออยู่กับผู้ให้บริการโดยตรง

3. การใช้งาน Intermediary ในลักษณะ Value-added ให้กับระบบ เช่น ใช้ Intermediary ในการเข้ารหัสข้อมูล (Encrypts) และเซ็นลายเซ็นอิเล็กทรอนิกส์ (Digital signs) ในการส่ง และรับ หรืออีกตัวอย่างหนึ่งเช่นต้องการเก็บข้อมูลเพิ่มเติมนำไปวิเคราะห์การทำงานของระบบโดยการรับ-

ส่งเอกสารจะผ่าน Intermediary แต่ไม่ได้ทำการประมวลผลเอกสารแต่จะเก็บข้อมูลบางอย่างไว้เท่านั้นเช่นนำไปวิเคราะห์ค่า QoS (Quality of Service) หรือ Bottleneck ของระบบ เป็นต้น

Intermediaries ใน SOAP

ในหัวข้อก่อนหน้านี้ได้อธิบายหลักการของการใช้ Intermediaries ไปแล้ว ดังนั้นในส่วนนี้จะกล่าวถึงว่าจะทราบได้อย่างไรว่าเอกสาร SOAP นั้นใครจะเป็นผู้ประมวลผล (ซึ่งจะอยู่ในส่วนของ <Body> Element ของเอกสาร)

ดังรูปที่แสดงการทำงานของ Intermediaries จะเห็นว่าเอกสาร SOAP จะถูกส่งจากผู้ขอใช้บริการผ่าน Intermediaries เป็นไปจนถึงปลายทางคือผู้ให้บริการ ดังนั้นเอกสาร SOAP ก็ถูกเปิดเผยทั้งหมดกับ Intermediaries ด้วย แต่ส่วนที่ Intermediaries สนใจจะอยู่เฉพาะส่วนที่เป็น Header เท่านั้น โดยใน Header จะมี Attribute หนึ่งเพื่อใช้ในการระบุว่าใครที่ควรจะทำงานกับ Header นี้ ซึ่ง Attribute นั้นก็คือ actor โดยค่าที่ actor จะต้องใส่ก็คือ URI (Universal Resource Indicator) ของ Intermediary นั้นเอง ดังตัวอย่าง

```
<soap:Header>
  <i:Authentication
    xmlns:i="http://www.i3t.or.th/ws/security"
    soap:actor="urn:I3T:SecurityGateWay"
    soap:mustUnderstand="1">
    <i:username>ACompany</i:username>
    <i:password>SOAPpath</i:password>
  </i:Authentication>
</soap:Header>
```

ดังนั้นเมื่อ Intermediary รับเอกสาร SOAP โดยถ้ามีชื่อ attribute actor ตรงกับของตัวเอง Intermediary จะเปลี่ยนเนื้อหาใน Header ที่ได้เพื่อส่งต่อไปยัง Intermediary อื่นหรือไปยังผู้ให้บริการจุดประสงค์หลักที่มีการออกแบบการทำงานแบบนี้ เพื่อลดความซับซ้อนของเอกสารลง นอกจากนี้แล้วยังมี URI ของ attribute actor ชนิดพิเศษอีกหนึ่งค่าคือ "http://schemas.xmlsoap.org/soap/actor/next" เพื่อบอกว่าผู้รับเอกสาร SOAP ในจุดต่อไปจะเป็นผู้ที่ประมวลผลของเอกสาร ซึ่งกระบวนการนี้จะมีประโยชน์อย่างมากในการส่งเอกสาร SOAP เป็นทอดๆ (hop-by-hop)

2.2.9. WSDL(Web Services Description Language)

WSDL (Web Services Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการใช้บริการของ Web Services และวิธีการติดต่อกับ Web Services ความต้องการของนิยามนี้เกี่ยวเนื่องกับความต้องการของ distributed system ที่จะกำหนด Interface Definition Language(IDL) โดยใช้ภาษา XML, WSDL เกิดจากการรวมแนวคิดของ NASSL (The

Network Accessible Service Specification Language), WDS (Well-Defined Services) ของบริษัทไอบีเอ็ม, SDL (The Service Description Language) และ SCL (the SOAP Contract Language) ของบริษัทไมโครซอฟท์ ปัจจุบัน WSDL เป็นภาษา ที่อยู่ในการดูแลของ W3C (World Wide Web Consortium) ซึ่งยังไม่เป็นมาตรฐานที่สมบูรณ์ เวอร์ชันที่ใช้งานอยู่ในปัจจุบันคือ WSDL 1.1 (รายละเอียดเพิ่มเติมเกี่ยวกับ WSDL สามารถศึกษาเพิ่มเติมได้ที่ <http://www.w3c.org/TR/wsdl>)

WSDL คือ มาตรฐานสำหรับการประกาศโปรเซสที่จำเป็นในการเรียกใช้เซอร์วิส SOAP (Simple Object Access Protocol)

โครงสร้างเอกสาร WSDL

WSDL เป็นภาษาที่อยู่ในความดูแลขององค์กร W3C (World Wide Web Consortium) version ที่มีอยู่ในปัจจุบัน คือ WSDL 1.1 ในการใช้งานจริง หากเราสร้างบริการเว็บเซอร์วิสก็จะมีเครื่องมือช่วยสร้างเอกสาร WSDL สำหรับเว็บเซอร์วิสอย่างอัตโนมัติ จุดภายในเอกสารที่เราควรรู้เกี่ยวกับการติดต่อและเรียกใช้บริการของเว็บเซอร์วิส มีจุดที่ควรรู้ ดังนี้

Element	Definition
<portType>	เป็นส่วนที่สำคัญที่สุดใน WSDL element อธิบาย operations ที่ web service มีให้บริการและ messages ที่เกี่ยวข้อง เทียบได้กับ function library หรือ module หรือ class ในการเขียนโปรแกรม
<operation>	อธิบาย method ที่ให้บริการ Web Services หนึ่งจะมี method จำนวนกี่ method ก็ได้
<message>	อธิบาย data elements ของ operation แต่ละ message อาจมีมากกว่าหนึ่งส่วนเทียบได้กับ parameter ของ function ในการเขียนโปรแกรม
<types>	อธิบายชนิดข้อมูลที่ web service ใช้ เพื่อความเป็นกลาง WSDL ใช้ XML Schema syntax ในการระบุชนิดข้อมูล
<binding>	อธิบาย format ของ message และ protocol details ในแต่ละ port
<service>	สำหรับ web server จะมี Web Services จำนวนกี่บริการก็ได้ และ ชื่อ Web Services ก็เป็นตัวจำแนกและบ่งบอกแต่ละบริการซึ่งห้ามมีชื่อซ้ำกัน

ตาราง 2.5 ตารางแสดงโครงสร้างของ WSDL

ตามทฤษฎีแล้ว ไฟล์เอกสาร WSDL แต่ละไฟล์ สามารถอธิบายคุณลักษณะของบริการเว็บเซอร์วิสได้มากกว่า 1 บริการ โดยแต่ละเว็บเซอร์วิสจะมีพอร์ตสื่อสารเฉพาะตัว ซึ่งบ่งบอกไว้ในเอกสาร WSDL อยู่แล้ว

WSDL Example

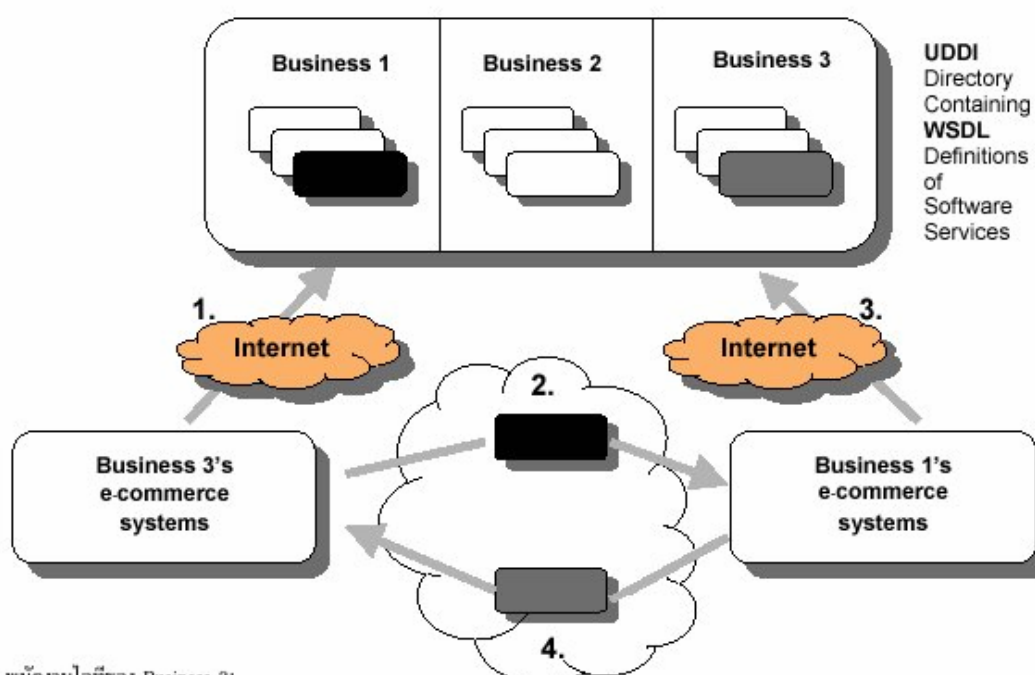
```

<wsdl:definitions name="nmtoken"? targetNamespace="uri">
  <import namespace="uri" location="uri"/> *
  <wsdl:documentation .... /> ?
  <wsdl:types> ?
    <wsdl:documentation .... /> ?
    <xsd:schema .... /> *
  </wsdl:types>
  <wsdl:message name="ncname"> *
    <wsdl:documentation .... /> ?
    <part name="ncname" element="qname"? type="qname"?/> *
  </wsdl:message>
  <wsdl:portType name="ncname"> *
    <wsdl:documentation .... /> ?
    <wsdl:operation name="ncname"> *
      <wsdl:documentation .... /> ?
      <wsdl:input message="qname"> ?
        <wsdl:documentation .... /> ?
      </wsdl:input>
      <wsdl:output message="qname"> ?
        <wsdl:documentation .... /> ?
      </wsdl:output>
      <wsdl:fault name="ncname" message="qname"> *
        <wsdl:documentation .... /> ?
      </wsdl:fault>
    </wsdl:operation>
  </wsdl:portType>
  <wsdl:serviceType name="ncname"> *
    <wsdl:portType name="qname"/> +
  </wsdl:serviceType>
  <wsdl:binding name="ncname" type="qname"> *
    <wsdl:documentation .... /> ?
    <!-- binding details --> *
    <wsdl:operation name="ncname"> *
      <wsdl:documentation .... /> ?
      <!-- binding details --> *
      <wsdl:input> ?
        <wsdl:documentation .... /> ?
        <!-- binding details -->
      </wsdl:input>
      <wsdl:output> ?
        <wsdl:documentation .... /> ?
        <!-- binding details --> *
      </wsdl:output>
      <wsdl:fault name="ncname"> *
        <wsdl:documentation .... /> ?
        <!-- binding details --> *
      </wsdl:fault>
    </wsdl:operation>
  </wsdl:binding>
  <wsdl:service name="ncname" serviceType="qname"> *
    <wsdl:documentation .... /> ?
    <wsdl:port name="ncname" binding="qname"> *
      <wsdl:documentation .... /> ?
      <!-- address details -->
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>

```

2.2.10. UDDI(Universal Description, Discovery and Integration protocol)

UDDI (Universal Description, Discovery, and Integration) เป็นมาตรฐานที่ให้ชุดพื้นฐาน APIs (Application Programming Interface) ของ SOAP ที่สามารถนำมาใช้ในการพัฒนาเป็นตัวแทนของผู้ให้บริการ (Service broker) UDDI ใช้สำหรับค้นหาเซอร์วิสที่ต้องการและเมื่อได้มาแล้ว UDDI ยังจัดหาข้อตกลงในวิธีการที่จะใช้งานเปรียบได้กับสมุดหน้าเหลือง เป็นมาตรฐานที่จัดตั้งขึ้นโดยบริษัทไอบีเอ็ม บริษัทไมโครซอฟต์ และบริษัทอริบา (Ariba) ปัจจุบันมีบริษัทที่ร่วม กันกำหนดมาตรฐานของ UDDI มากกว่า 70 บริษัท ซึ่งมาตรฐานของ UDDI ถูกกำหนดให้เป็นมาตรฐานสำหรับ B2B interoperability (รายละเอียดเพิ่มเติมเกี่ยวกับ UDDI สามารถศึกษาเพิ่มเติมได้ที่ <http://uddi.org>)



พนักงานไอทีของ Business 3:

1. ค้นหาข้อกำหนดการให้บริการใน WSDL ผ่านอินเทอร์เน็ตของ Business 1
2. ติดต่อผ่านอินเทอร์เน็ตของแอปพลิเคชันที่ให้บริการ (ด้วย SOAP) ผ่านทางอินเทอร์เน็ต
- 3,4. Business 1 รวมหรือบูรณาการกับ Business 3 ด้วยวิธีเดียวกัน

รูป 2.4 Web Services with SOAP, UDDI และ WSDL

2.3. .NET เทคโนโลยี

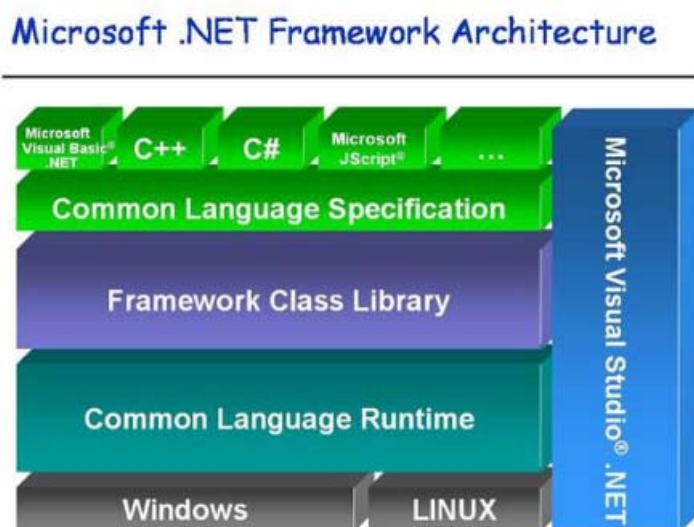
2.3.1. ความหมายของ .NET

.NET (อ่านว่า คอตเน็ต) คือแนวคิดหนึ่งที่บริษัทไมโครซอฟท์ภูมิใจนำเสนอ โดย .NET ตัวนี้ไม่ได้เกี่ยวข้องกับโดเมนเนมของเว็บใด ๆ ทั้งสิ้น แต่ .NET ตัวนี้ หมายถึง การนำเอาอุปกรณ์ทุกอย่างบนโลกมาเชื่อมโยงต่อกันเหมือนตาข่าย (net = ตาข่าย) ซึ่งหากว่าทำสำเร็จแล้ว ไม่ต้องนึกเลยว่าไมโครซอฟท์จะได้เป็นเจ้าแห่งเทคโนโลยีอย่างไม่ต้องสงสัย แต่เรื่องมันไม่ง่ายอย่างที่คิด เพราะอุปกรณ์ต่างเหล่านั้นล้วนถูกออกแบบมาต่างหาก การที่มันจะติดต่อสื่อสารกันรู้เรื่องนั้น ย่อมเป็นเรื่องที่เป็นไปได้ยาก บริษัทไมโครซอฟท์เล็งเห็นจุดนี้ จึงได้พยายามที่จะคิดค้นสิ่ง ที่เป็นมาตรฐานขึ้น เพื่อให้อุปกรณ์ทุกชนิดทั่วโลกติดต่อสื่อสารกันได้อย่างรู้เรื่อง จึงไม่ใช่เรื่องแปลกถ้าในอนาคตเราเปิดเว็บไซต์เล่นอินเทอร์เน็ตด้วยอุปกรณ์อื่นๆนอกเหนือจากคอมพิวเตอร์

2.3.2. .NET Framework

จากที่กล่าวข้างต้นบริษัทไมโครซอฟท์ต้องการที่จะสร้างอะไรที่เป็นมาตรฐานขึ้นมา เพื่อให้ทุกสิ่งทุกอย่าง เพื่อให้ทุกสิ่งทุกอย่างสามารถติดต่อสื่อสารกันได้หมด โดยคิดค้นระบบ ซึ่งหมายมั่นปั้นมือว่าจะให้เป็นระบบมาตรฐาน ระบบนี้คือ .NET Framework ซึ่งระบบนี้ไม่ใช่ระบบปฏิบัติการ แต่เปรียบเสมือนโปรแกรมหนึ่งที่จะสามารถสร้างสภาวะแวดล้อมหนึ่ง ซึ่งสามารถทำงานในระบบ .NET นี้ได้

ในอนาคตบริษัทไมโครซอฟท์ก็มีแผนที่จะนำเอาระบบนี้ไปติดตั้งลงบนอุปกรณ์ทุกชนิด เพื่อให้ทุกสิ่งทุกอย่างมีระบบๆหนึ่งที่เหมือนกันหมดโดย .NET Framework นั้นมีส่วนประกอบภายในแบ่งออกเป็น 3 ชั้นใหญ่ๆคือ



รูป 2.5 แสดง โครงสร้างของ .NET Framework

- 1) Programming Language: เป็นรูปแบบของภาษาที่ออกแบบมาเพื่อให้สามารถทำงานในสภาวะที่เป็น .NET ได้โดยที่ทางบริษัทไมโครซอฟท์ได้เปิดตัวภาษาหลักๆที่จะใช้พัฒนานบน .NET นี้ 3 ภาษา
 - C# เป็นภาษาใหม่ที่บริษัทไมโครซอฟท์พัฒนามาจาก C++ กับ JAVA เป็นหลัก
 - VB.NET เป็นภาษาที่พัฒนามาจาก Visual Basic ในเวอร์ชัน 6.0
 - JScript.net เป็นภาษาที่พัฒนามาจาก JScript ซึ่งเป็น JavaScript ในเวอร์ชันของบริษัทไมโครซอฟท์
- 2) Base Classes Library: Library นั้นเปรียบเสมือนชุดคำสั่งสำเร็จรูปย่อยๆที่เพิ่มเข้ามา ซึ่งส่วนใหญ่จะเป็นชุดคำสั่งที่ต้องใช้งานอยู่เป็นประจำ ดังนั้นจึงมีผู้คิดค้นเพื่ออำนวยความสะดวกในการเขียนโปรแกรม ซึ่ง Library ในภาษาต่างๆส่วนใหญ่จะอยู่ในรูปแบบไฟล์ include แต่ถ้าเป็น ASP สิ่งที่เป็น library ก็คือ คอมโพเนนท์ต่าง ๆ นั้นเอง ซึ่งภายในระบบ .NET จะสร้างสิ่งที่เรียกว่าเป็น Library พื้นฐานขึ้น ทำให้ไม่ว่าจะใช้ภาษาใดในการพัฒนาโปรแกรมก็สามารถที่จะเรียกใช้ Library ที่เป็นตัวเดียวกันได้หมด
- 3) Common Language Runtime (CLR): นับเป็นสิ่งสำคัญแทบจะที่สุดของระบบ .NET นี้ก็ได้ เพราะ CLR ที่ว่านี้มีหน้าที่ทำให้โปรแกรมที่เขียนขึ้นมาด้วยภาษาต่าง ๆ กัน กลายเป็นภาษารูปแบบมาตรฐานเดียวกัน ทั้งหมด เราเรียกภาษาที่ว่านี้ว่า Intermediate language (IL) ซึ่งเมื่อต้องการที่จะรันโปรแกรมใด CLR ที่ว่านี้จะตรวจสอบเครื่องที่รันว่ามีสภาวะแวดล้อมการทำงานเช่นใดหลังจากนั้นก็จะได้คอมไพล์เป็นโปรแกรมที่เหมาะสมต่อการทำงานของเครื่องนั้น ทำให้เราสามารถใช้งานโปรแกรม ต่างๆได้อย่างมีประสิทธิภาพสูงสุดในแต่ละเครื่อง

2.3.3. ข้อดีของ .NET Framework

- 1) ประโยชน์และข้อดีของ .NET Framework นั้นพอจะสรุปออกมาได้เป็นข้อๆดังนี้
- 2) เป็นระบบที่มีไลบรารีที่เป็นมาตรฐานเดียวกัน: เนื่องจากมีไลบรารีที่เป็นมาตรฐานเดียวกันทั้งหมดทำให้เราไม่ต้องกังวล ว่าภาษาที่ใช้เขียนนั้นมีไลบรารีตัวนั้นตัวนี้หรือไม่ รวมทั้งไม่ต้องคอยกังวลว่าถ้าใช้ไลบรารีของภาษาหนึ่งแล้วอีกภาษาหนึ่งจะไม่มีไลบรารีตัวนั้น
- 3) ไม่ขึ้นกับระบบปฏิบัติการ (OS): เนื่องจากระบบปฏิบัติการ ที่แต่ละบุคคลหรือองค์กรใช้นั้นย่อมไม่เหมือนกัน แต่ภายใน .NET Framework จะไม่มีปัญหานี้ของ

เพียงแค่มีระบบ .NET Framework ก็จะทำให้สามารถใช้งานโปรแกรมต่างๆ ได้ ซึ่งเป็นข้อดีตรงที่เราจะสามารถใช้โปรแกรมต่างๆ ได้ทุกระบบปฏิบัติการ

- 4) ใช้ในการพัฒนาได้ทุกภาษา: ทำให้เราไม่ต้องคอยมาศึกษาภาษาใหม่ๆ เมื่อต้องการสร้างโปรแกรมในแต่ละครั้ง นอกจากนั้นเรายังสามารถเลือก ใช้ภาษาที่เราถนัดที่สุดในการพัฒนาโปรแกรมต่างๆ ได้ด้วย
- 5) มีการควบคุมสิ่งแวดล้อมในการทำงานเป็นอย่างดี :เนื่องจากเป็นระบบที่เป็นมาตรฐานทำให้การควบคุมจัดสรรระบบต่างๆ ทำได้ง่ายขึ้น ไม่ว่าจะเป็นการจัดสรรหน่วยความจำ ด้านการใช้งานเครื่องก็มีความรวดเร็วมากขึ้น ลดโอกาสที่เครื่องจะแฉงก็ได้เป็นอย่างดี
- 6) ความปลอดภัยที่มีมากขึ้น: .NET Framework สามารถกำหนดสิทธิการใช้งานหรือ permission ของผู้ใช้งานได้มากขึ้นทำให้สามารถกำหนดว่า จะให้โปรแกรมในส่วนใดใช้งานได้หรือไม่ได้ แล้วแต่เฉพาะบุคคล

2.4. ASP.NET

2.4.1. ความเปลี่ยนแปลงที่เกิดขึ้นของ ASP.net

เราสามารถเลือกสร้างแอปพลิเคชัน ASP.net ได้ทันทีจาก Visual Studio.net ซึ่งการสร้าง

แอปพลิเคชันประเภทนี้ นอกจากจะสนับสนุนการทำงานบนอุปกรณ์ต่างๆ เช่น บราวเซอร์บนเครื่อง พีซี ,PDA ,Moblie phone แล้ว ตัว ASP.net มีสิ่งที่มีการเปลี่ยนแปลงมากทีเดียว

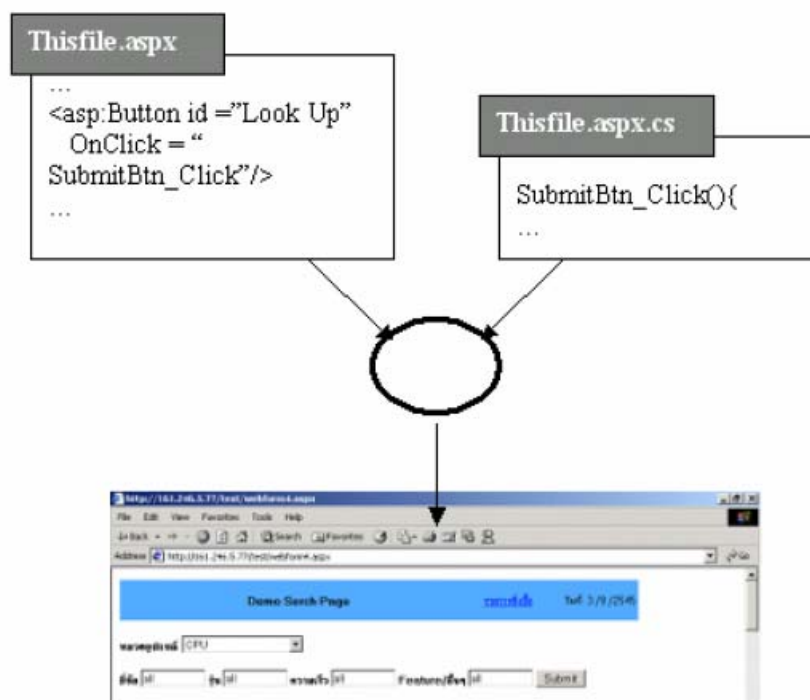
2.4.2. นามสกุลของไฟล์ และการแยกส่วนออกแบบกับส่วนเขียนโค้ด

เริ่มจากนามสกุลของไฟล์ที่สร้าง จะเปลี่ยนจาก .asp เป็น .aspx และ 1 หน้าของเว็บเพจของไฟล์ .aspx

ประกอบด้วย 2 ส่วน คือ

1. ไฟล์ที่มีนามสกุล .aspx
2. ไฟล์ที่มีนามสกุลในแบบ Visual c# หรือ Visual Basic

เป็นการแยกระหว่างการออกแบบหน้าเว็บเพจกับการเขียนโปรแกรม ปัจจุบันการเขียนเว็บเพจจะรวมโค้ดทั้งหมดไว้ ไม่ว่าจะเขียนด้วย VBScript หรือ JavaScript และโครงสร้างของภาษา HTML จะรวมกันที่ไฟล์ ASP อย่างเดียวกัน นี่ทำให้ประสบปัญหา มาก แต่ไฟล์ .aspx จะแยกออกมา 2 ส่วนตามที่กล่าวมาแล้ว ทำให้การพัฒนาโปรแกรม เป็นเรื่องง่ายขึ้น



รูป 2.6 แสดงเว็บแอปพลิเคชันที่สร้างด้วย ASP.net จะแยกไฟล์สำหรับออกแบบ(file.aspx) กับไฟล์
สำหรับเขียน โค้ด โปรแกรมจากกัน(file.aspx.cs)

2.4.3. การคอมไพล์โปรแกรม

ไฟล์ที่สร้างขึ้นจาก ASP.net เมื่อถึงตอนทำงาน คือ เมื่อทำการคอมไพล์แอปพลิเคชัน ทั้ง 2 ไฟล์จะถูกนำมารวมกัน และคอมไพล์ออกมาเป็น Common Language Runtime จากนั้นเมื่อมีการเรียกใช้งาน ก็จะถูกคอมไพล์เป็น ไบนารีโค้ดและนำไปรันเพื่อแสดงผลได้

ส่วนเว็บเพจที่เป็น ASP แบบเดิม จะมี ASP เป็นตัวแปลภาษา (interpret) คือทุกครั้ง ที่เว็บเพจจะถูกนำไปแสดงผล ก็จะต้องคอมไพล์ พอผู้ใช้คนที่ 2 เข้ามาเรียกเว็บเพจ ก็จะถูกคอมไพล์อีก ซึ่งต่างจากASP.net

โครงสร้างของไฟล์ .aspx ในบรรทัดแรก จะมีส่วนที่เป็นตัวบอกว่าอีกไฟล์หนึ่งเป็น ไฟล์เขียนโค้ดโปรแกรม นั้น เป็นภาษาอะไร ไฟล์ชื่ออะไร ซึ่งเป็นการลิงค์ระหว่างไฟล์ Visual Basic หรือ Visual C# ที่ทำงานอยู่เบื้องหลังแอปพลิเคชันนี้

```
<%@Page language="vb"
Codebehind = "events_II.aspx.vb"%>
```

นอกจากนั้นยังมีแท็กคำสั่งพิเศษคือ runat "server " เพื่อบอกให้ทราบว่าการทำงานของโปรแกรมจะอยู่บนเซิร์ฟเวอร์

```
<Form runat="server"> . . . </Form>
```

2.4.4. คอนโทรลที่ใช้ออกแบบในแอปพลิเคชัน ASP.net

การเขียนโปรแกรมใน ASP.net จะใช้ภาษาใดก็ได้ใน .net ทั้งภาษาหลักใน Visual Studio.net อย่าง Visual C# , Visual Basic .net หรือภาษาอื่นๆ ที่สนับสนุน .net เช่น ภาษา Pascal ก็สามารถเขียนได้ โดยแอปพลิเคชัน ที่สร้างขึ้นนี้สามารถรันได้บนทุกอุปกรณ์ที่สนับสนุน .net

แอปพลิเคชันที่สร้างด้วย ASP.net นั้น จะเป็นการแยกระหว่างส่วนที่ใช้เขียนโค้ดโปรแกรม กับส่วนที่ใช้ออกแบบออกจากกัน โดยฟอร์มที่อยู่ใน ASP.net ก็จะคล้ายๆ กับ Windows Form ซึ่งคอนโทรล 3 ประเภทที่ใช้ในการสร้าง ASP.net มีดังนี้

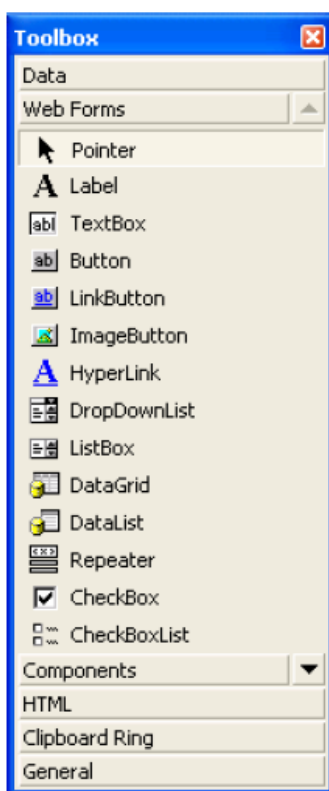
2.4.5. คอนโทรล HTML

คอนโทรล HTML นั้นจะมีโครงสร้างเหมือนเดิมที่ใช้บน ASP แต่เพิ่มบรรทัด Runat = "server" ขึ้นมา ก็บอกแล้วว่าคอนโทรลนี้ใช้กับ ASP.net เช่นเดียวกันหากในฟอร์มนี้ต้องการให้คอนโทรลบางตัวไม่ทำงานในแบบของ ASP.net เพียงแต่เอาคำว่า Runat "server" ออกเท่านั้นเอง

(System.Web.UI.HtmlControls)

<input type=button id=button1

OnServerClick = "Button1_Click" runat="server"/>



รูป 2.7 คอนโทรล HTML ที่เราสามารถเลือกสร้างได้จาก Tool Box

2.4.6. คอนโทรล Web Forms

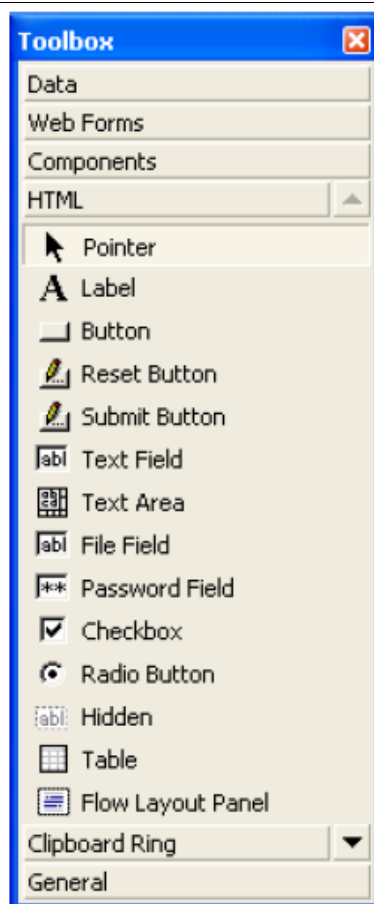
คอนโทรล Web Forms เป็นคอนโทรลพิเศษ ที่มีเฉพาะใน ASP.net คอนโทรลพวกนี้ จะมีคำว่า ASP อยู่ข้างหน้าและมีคำว่า Runat = "Server"

(System.Web.UI.WebControls)

```
<asp: label id= " label " runat ="server">
```

```
backcolor = "red" ForeColor ="white">
```

```
Hello World </label>
```



รูป 2.8 คอนโทรล Web Forms ที่เราสามารถเลือกสร้างได้จาก Tool Box

2.4.7. คอนโทรล Customer

คอนโทรลนี้มีนามสกุลเป็น .ascx เหมือน ActiveX control บน Windows Forms นั่นเอง

2.4.8. คอนโทรลที่ทำงานฝั่งเซิร์ฟเวอร์

ตัวคอนโทรลที่ใช้สำหรับออกแบบบนเว็บเพจที่ทำงานทางฝั่งเซิร์ฟเวอร์นั้น มีกว่า 45 คอนโทรลตั้งแต่คอนโทรลมาตรฐานอย่าง TextBox, CheckBox จนถึงคอนโทรลพิเศษที่พัฒนาขึ้นมาเพื่อใช้กับASP.net

ตัวอย่างโค้ดของคอนโทรลพื้นฐานที่ทำงานบนเซิร์ฟเวอร์

```
<asp:textbox id =text1 runat=server/>
text1.text ="Hello World"
<asp:checkbox id=check1 runat=server/>
check1.checked=True
<asp:button id=button1 runat=server/>
button1_onClick()
<asp:DropDownList id=DropDownList1 runat=server>
DropDownList1.SelectedItem.Text = "Hello"
```

คอนโทรลพื้นฐานมีวิธีการใช้งานเหมือนกับคอนโทรล HTML ทั่วไป เพียงแต่เพิ่มคำว่า ASP และ : (โคลอน) ตามด้วยชนิดของคอนโทรล กับคำว่า runat = "server" รวมทั้งการอ้างถึงค่าอื่นๆ เช่นข้อความที่ต้องการให้แสดงบนคอนโทรล

ตัวอย่างของคอนโทรลพิเศษเช่น Data Grid ใช้แสดงข้อมูล สามารถรวมเข้ากับ ADO เพื่อนำข้อมูลจากฐานข้อมูลมาแสดง นอกจากนี้มีคอนโทรลประเภท Validation เพื่อตรวจสอบการป้อนข้อมูลของเรา

คอนโทรลประเภท Validation เป็นคอนโทรลที่เราพบบ่อยๆ เช่น การสร้างฟอร์มให้ผู้ใช้งานกรอกข้อมูล ในปัจจุบันหากเราสร้างคอนโทรลประเภทนี้เอง คงต้องมาทำการเขียนโค้ดเอง แต่สิ่งที่เกิดในASP.net คือมีคอนโทรลชื่อ RequiredFieldValidator ให้เราสามารถตรวจสอบได้

```
<asp:RequiredFieldValidator ControlToValidate ="txtName"
ErrorMessage="Please Enter Your Name" runat ="server"/>
```

โดยเราสามารถกำหนดพร็อพเพอร์ตี้ที่ Error message หากไม่มีการกรอกข้อมูล ให้แสดงข้อความที่อยู่ต่อท้ายพร็อพเพอร์ตี้

2.4.9. การเขียนโปรแกรมใน ASP.net

ASP นั้นมีการเขียนโปรแกรมเพื่อดักอีเวนต์ (Event Driven) ด้วย VBScript หรือ JavaScript โดยจะมี Event Driven 4 ตัวดังนี้

- Application_OnStart เป็นอีเวนต์ที่เกิดขึ้น เมื่อมีการเรียกใช้งานเว็บเพจใดๆ ก็ได้ในเว็บไซต์เป็นเว็บเพจแรก
- Application_OnEnd เกิดขึ้นเมื่อมีการปิดเครื่องเซิร์ฟเวอร์ หรือทำการ Stop Service ของเว็บ
- เซิร์ฟเวอร์
- Session_OnStart เกิดขึ้นเมื่อมีการเปิดเครื่องเซิร์ฟเวอร์ หรือทำการ Stop Services ของเว็บ
- เซิร์ฟเวอร์
- Session_OnEnd เกิดขึ้นเมื่อไคลเอนต์จบการทำงานบนเว็บไซต์

2.4.10. สรุปตารางเปรียบเทียบระหว่าง ASP กับ ASP.net

คุณลักษณะ	ASP	ASP.NET
นามสกุลของไฟล์	.asp	.aspx , .ascx , .asmx
การทำงานของโปรแกรม	คอมไพล์โค้ดด้วยตัว Interpreter	คอมไพล์โค้ดด้วยตัว JIT Compiler
การออกแบบและการเขียนโปรแกรม	รวมส่วนการเขียนโปรแกรมและส่วนการออกแบบด้วยกัน	แยกส่วนการเขียนโปรแกรมและส่วนการออกแบบออกจากกัน
การจัดการเรื่องสถานะของโปรแกรม	มีการใช้ Cookies ที่ไคลเอนต์ไม่สามารถทำงานร่วมกันผ่านแพลตฟอร์มได้	สามารถฝังลง URL ได้เลยโดยไม่ต้องใช้ Cookies สามารถบันทึกข้อมูลที่ใช้ร่วมกันลงในฐานข้อมูล SQL ภายนอกได้
อัปเดตไฟล์ DLL Library	ต้องปิดเว็บเซิร์ฟเวอร์ก่อน	สามารถคัดลอกไฟล์ DLL ลงในไดเรกทอรี bin ได้ทันที (ไม่ต้องการลงทะเบียน)

ตาราง 2.6 แสดงตารางเปรียบเทียบระหว่าง ASP กับ ASP.net

2.5. ภาษา C#

2.5.1. โครงสร้างพื้นฐาน

1) Comment

เป็นข้อความที่ถูกแทรกเข้าไปในโปรแกรม เพื่ออธิบายให้ผู้อ่านสามารถเข้าใจโปรแกรมนั้นได้ง่ายขึ้นหรือเพื่อบันทึกข้อมูลที่เกี่ยวข้องกับโปรแกรมนั้น ข้อความที่เป็น comment จะไม่มีผลต่อการทำงานของโปรแกรม ภาษา C# มี comment อยู่ 3 แบบ

- เหมือนภาษา C คือ เริ่มต้นด้วย /* และจบด้วย */ แบบนี้สามารถ comment ได้หลายบรรทัด เช่น `<font color="#006600" face="Courier New">/* This is multiple<br> line comment */`
- แบบเดียวกับ C++ คือเริ่มต้นด้วย // ไปจนจบบรรทัดนั้น โดยไม่ต้องมีตัวปิด
- แบบที่สามขึ้นต้นด้วย /// ข้อความในนี้จะถูกนำไปทำเป็น xml documentation

2) Keyword

คือคำ ที่ถูกกำหนดความหมายไว้แล้ว ในภาษาส่วนใหญ่นั้น รวมทั้ง C# จะถือว่า keyword เป็น reserved word (คำสงวน) นั่นคือไม่สามารถนำไปใช้ในความหมายอื่น นั่นคือห้ามนำไปใช้ เป็นชื่อตัวแปร

ชื่อฟังก์ชัน หรือชื่อคลาส ภาษา C# มี keyword ทั้งหมดดังนี้

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	goto
if	implicit	in	int	interface
internal	is	lock	long	namespace
new	null	object	operator	out
override	params	private	protected	public
readonly	ref	return	sbyte	sealed
short	sizeof	stackalloc	static	string
struct	switch	this	throw	true
try	typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual	void
volatile	while			

ตาราง 2.7 ตารางแสดงชื่อคลาสในภาษา C#

C# คล้ายกับ Java ตรงที่รวมเอาคำว่า true, false และ null ซึ่งถือว่าเป็นค่าคงที่ เข้ามาเป็น keyword

3) Identifier

คือชื่อที่ผู้เขียนโปรแกรมตั้งขึ้นได้ อาจเป็นชื่อของคลาส ตัวแปร ฟังก์ชัน ชื่อที่ถูกต้องจะต้องไม่เป็น keyword อาจจะประกอบด้วย ตัวอักษรตัวใหญ่, ตัวเล็ก, ตัวเลข และ underscore identifier ที่ถูกต้อง เช่น ThisIsABook, num1, _x, System, Int32 โดยชื่อที่ถูกต้องห้ามขึ้นต้นด้วยตัวเลข และจะมีความยาวเท่าไรก็ได้ ภาษา C# นั้นเป็นภาษาที่ case-sensitive นั่นคือ ตัวเล็ก กับ ตัวใหญ่จะตีความไม่เหมือนกัน เช่น WriteLine กับ writeline นั้นเป็นคนละตัวกัน

Verbatim identifier

verbatim identifier คือ identifier ที่นำเอา keyword มาตั้งชื่อได้ โดยการเติมตัวอักษร @ เข้าไปข้างหน้า

Example :

```
class @class {
    public static void @static(bool @bool) {
        if (@bool)
            System.Console.WriteLine("true");
        else
            System.Console.WriteLine("false");
    }
}
```

หลักการตั้งชื่อ Identifier ที่ดีคือ

- ตั้งให้สื่อความหมาย
- ใช้ตัวเล็กตัวใหญ่เพื่อแบ่งคำ
- ถึงแม้ การตั้งชื่อด้วยคำเดียวกันแต่ต่างกันที่ตัวอักษรใหญ่ เล็ก ต่างกันจะทำได้ แต่เราก็ไม่ควร เนื่องจาก .NET นั้นสามารถเรียกใช้ คลาสหรือฟังก์ชันข้ามภาษากันได้ ดังนั้นภาษาที่ case-insensitive เช่น VB.NET อาจจะไม่สามารถใช้งาน identifier แบบนี้ได้

4) Literal

คือลำดับของตัวอักษรที่ใช้แสดงค่าคงที่ใน source code แบ่งออกเป็น

- **Integral literal** คือค่าคงที่ของเลขจำนวนเต็ม เขียนได้ 3 แบบ
 - ตัวเลขล้วน ๆ ที่ไม่ขึ้นต้นด้วยเลข 0 จะถูกตีความเป็นเลขฐาน 10
 - ตัวเลข 0-7 ที่ขึ้นต้นด้วยเลข 0 จะถูกตีความเป็นเลขฐาน 8 เช่น 0400 คือ 256 ในเลขฐาน 10
 - ตัวเลข 0-9 และอักษร A-F(หรือ a-f) ที่ขึ้นต้นด้วย 0X(หรือ 0x) จะถูกตีความเป็นเลขฐาน 16 เช่น 0X7F คือ 127 ในเลขฐาน 10

โดย default แล้ว Integral literal จะถูกเก็บไว้ในหน่วยความจำขนาด 32-bit(ค่าคงที่ชนิด int) หากเราต้องการใช้ตัวเลขที่มากกว่านั้นคือตั้งแต่ 231 จนถึง $2^{63} - 1$ จะต้องมิตัวอักษร l(แอลเล็ก) หรือ L ต่อท้ายเลขนั้น เพื่อบอก compiler ว่าเป็นเลข long integer(ค่าคงที่ชนิด long)

- **Floating-point Literal** คือค่าคงที่เลขทศนิยม เขียนได้สองแบบ
 - แบบมาตรฐาน เช่น 17.357810
 - แบบวิทยาศาสตร์ เช่น $2.997E8(2.997 * 10^8)$ หรือ $9.1093897e-31$ (จะใช้ e หรือ E ก็ได้)

ในภาษา C# มี floating-point สองแบบคือแบบ 32-bit(ค่าคงที่ชนิด float) ซึ่งจะมีความละเอียดของทศนิยมประมาณ 7 หลัก และแบบ 64-bit(ค่าคงที่ประเภท double) โดย default ถ้าเราไม่ระบุตัวอักษรต่อท้าย compiler จะตีความเป็น ค่าคงที่ชนิด double ซึ่งเราควรที่จะกำหนดไปให้ชัดเจนว่าเป็น ตัวแปรขนาดใดโดยการเติมตัวอักษร f(หรือ F) เพื่อบอกว่ามีขนาด 32-bit หรือ d(หรือ D) เพื่อบอกว่ามีขนาด 64-bit

- **Boolean Literal** คือค่าคงที่ความจริง (truth value) มีค่าเป็น true หรือ false
- **Character Literal** คือค่าคงที่ตัวอักษร

โดยทั่วไปแล้ว จะถูกแสดงอยู่ใน single quote เช่น 'a' และ '0' หากเป็น อักษรพิเศษ เช่นพวก control character จะต้องถูกนำหน้าด้วย \ (backslash) โดยจะถูกเรียกว่า escaped character

- **String Literal** มีรูปแบบคล้าย ๆ กับ character ต่างกันที่จะถูกแสดงอยู่ใน double quote และการจะแสดงเครื่องหมาย single quote ไม่จำเป็นต้องใช้ escaped character

5) Operator

- **Assignment operator** ในภาษา C# นั้นก็ได้้นำความคิดมาจากภาษา C นั่นคือ เครื่องหมาย = เป็น operator ที่สามารถนำมาใช้ได้ ใน expression ปกติได้ เช่น $x + (y = 1)$ นั่นคือมีการกำหนดค่า y ให้เท่ากับ 1 ก่อนแล้วจึงนำไป + กับ x
- **Arithmetic operator** แบ่งย่อยออกเป็น
 - **Integer & Floating-point operator** ได้แก่ +, -, *, /, %(modulus) โดยถ้า operand ทั้งสองของ operator / เป็น integer number, operator / จะถูกตีความเป็น div(หารแบบปัดเศษ) แต่ถ้ามี operand ตัวใดตัวหนึ่งเป็น floating-point number, operator / จะถูกกระทำการแบบการหารปกติ
 - ในภาษา C# จะเหมือนกับ Java นั่นคือ modulus operator นั้นจะสามารถใช้กับ floating-point number ได้ด้วย
 - **Arithmetic assignment operator** ได้แก่ +=, -=, *=, /=, %= มีไว้ช่วยเขียน ประโยค assignmentให้อ่านง่ายขึ้น เช่น $x = x + 7$ สามารถเขียนได้เป็น $x += 7$ อ่านได้ว่า เพิ่มค่า x อีก 7
 - **Increment and Decrement operator** ได้แก่ operator 2 ตัว คือ ++ และ -- ซึ่งเขียนได้ทั้งแบบ prefix และ postfix โดย operator นี้ เป็น operator ที่ใช้เพิ่มหรือลดค่าตัวแปรอีกทีละ 1 โดยจุดเริ่มต้นของ operator นี้มาจากภาษา C เพราะคำสั่งประเภทนี้จะถูกแปลเป็นคำสั่ง INC, DEC ในภาษา assembly ซึ่งจะทำงานได้เร็วกว่า ADD และ SUB แต่ในการใช้งานจริงแล้ว ประโยชน์ของมันคือการที่ code อ่านง่ายขึ้น
- **Bitwise operator** เป็น operator ที่จะกระทำกับทุก ๆ bit ของ operand ในชนิด integral type(sbyte, short, int, long, byte, ushort, uint, ulong)
 - **Boolean bitwise operator** ได้แก่ ~(bitwise unary NOT), &(bitwise AND), |(bitwise OR), ^(bitwise XOR), <<(shift left), >>(shift right)
 - **Assignment bitwise operator** เช่นเดียวกับ Arithmetic assignment operator คือนำเอา Boolean bitwise operator มารวมกับเครื่องหมาย = ได้เป็น operator ดังนี้ ~=, &=, |=, ^=, <<=, >>=

- **Relational operator** ได้แก่ ==(equal to), !=(not equal to), >(greater than), >= (greater than or equal), <(less than), <=(less than or equal) ใช้ในการเปรียบเทียบกับ operand ที่มีประเภท ตัวเลข, ตัวอักษร(char) , ข้อความ(string), bool โดยข้อมูลประเภทที่ไม่ใช่ตัวเลขนั้น จะสามารถเปรียบเทียบโดยใช้ operator == และ != ได้เท่านั้น และต้องเปรียบเทียบกับข้อมูลประเภทเดียวกันเท่านั้น
โดยผลลัพธ์ของการเปรียบเทียบนั้นจะได้ออกมาค่า boolean(true กับ false)
- **Logical operator** เนื่องจากว่าในภาษา C# มี boolean type แล้ว ดังนั้น operator ในกลุ่มนี้ จึงสามารถกระทำกับ operand ที่เป็น boolean เท่านั้น (เช่นเดียวกับ Java)
 - **Boolean Logical operator** ได้แก่ &(logical AND), |(logical OR), ^(logical XOR) และ !(logical NOT) จะสังเกตได้ว่า operator เหล่านี้ จะใช้เครื่องหมายเดียวกับ bitwise operator โดย compiler จะดูจาก operand ว่าเป็นชนิดใด จึงจะตัดสินใจใช้ operator ในความหมายเหล่านั้น
 - **Short-Circuit Logical operator** ได้แก่ &&(short-circuit AND) และ ||(short-circuit OR) ต่างกับ Boolean Logical operator ตรงที่ว่า จะหยุดคำนวณเมื่อ ผลลัพธ์ของทั้ง expression นั้นสามารถระบุค่าได้แล้ว เช่น
- **Conditional operator** จริง ๆ ไม่อยากเขียนเรื่องนี้เลยเพราะยังทำความเข้าใจกับรูปประโยคค่อนข้างยาก เอาเป็นว่ามันคือ if clause แบบใช้สัญลักษณ์ละกัน
- **String operator** มี +, =, ==, != และ += โดย เครื่องหมาย + มีไว้ต่อ string โดยที่จะต่างกับ + ทั่ว ๆ ไปตรงที่ไม่มีคุณสมบัติสลับที่ นอกนั้นก็มีความหมายเหมือนปกติ

6) Primitive Data Type

Data type	Description	Aliased From
sbyte	1-byte signed integer เก็บค่าตั้งแต่ -128 ถึง 127	public struct System.SByte
byte	1-byte unsigned integer เก็บค่าตั้งแต่ 0 ถึง 255	public struct System.Byte
short	2-byte signed integer เก็บค่าตั้งแต่ -32768(-2^{15}) ถึง 32767($2^{15} - 1$)	public struct System.Int16

Data type	Description	Aliased From
ushort	2-byte unsigned integer เก็บค่าตั้งแต่ 0 ถึง $65536(2^{16} - 1)$	public struct System.UInt16
int	4-byte signed integer เก็บค่าตั้งแต่ $-2,147,483,648(-2^{31})$ ถึง $2,147,483,647(2^{31} - 1)$	public struct System.Int32
uint	4-byte unsigned integer เก็บค่าตั้งแต่ 0 ถึง $4,294,967,295(2^{32} - 1)$	public struct System.UInt32
long	8-byte signed integer เก็บค่าตั้งแต่ $-9,223,372,036,854,775,808(-2^{63})$ ถึง $-9,223,372,036,854,775,807(2^{63} - 1)$	public struct System.Int64
ulong	8-byte signed integer เก็บค่าตั้งแต่ 0 ถึง $18,446,744,073,709,551,616(2^{64} - 1)$	public struct System.UInt64
float	4-byte floating-point number เก็บค่าในช่วง $\pm(1.401298E-453.4028235E+38$ ถึง $3.4028235E+38)$	public struct System.Single
double	8-byte floating-point number เก็บค่าในช่วง $\pm(4.94065645841246544E-324$ ถึง $1.79769313486231570E+308)$	public struct System.Double
bool	Boolean type เก็บค่า true และ false	public struct System.Boolean
char	2-byte Unicode character	public struct System.Char

ตาราง 2.8 แสดง Primitive Datatype

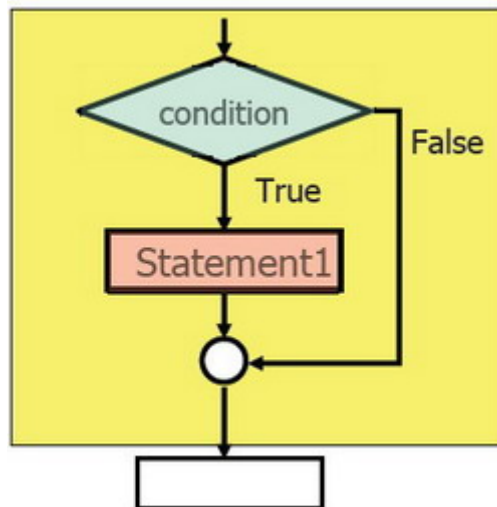
7) Data type อื่น ๆ ที่ใช้บ่อย

Data type	Description	Aliased From
decimal	floating-point แบบละเอียด เก็บค่าตั้งแต่ $(-2^{96}$ ถึง $2^{96})/10^{(0 \text{ ถึง } 28)}$ เวลาจะระบุค่า literal ใด ๆ ว่าเป็น decimal ให้ใช้ ตัว M หรือ m ต่อท้าย เช่น 0.001m	public struct System.Decimal
DateTime	เก็บค่าข้อมูล วันที่และเวลาในช่วง 00:00:00 1 มกราคม 0001 จนถึง 23:59:59 31 ธันวาคม 9999 โดยที่มีความละเอียดในการเก็บ 0.1 millisecond(100 nanosecond)	public struct System.DateTime
string	ลำดับของ character ที่ถูก encode แบบ UTF-16	public class System.String
object	เป็น type ที่ใหญ่ที่สุดในนี้คือทุก ๆ type สามารถเปลี่ยนมาเป็น type นี้ได้หมด (คล้าย ๆ กับ variant ใน VB®) ละครับ (แต่ประหัต mem และ พลังงานในการใช้กว่าเยอะ)	public class System.Object

ตาราง 2.9 แสดง Primitive Datatype อื่น ๆ ที่ใช้บ่อย

2.5.2. Control Statement

1) If..Clause



รูป 2.9 แสดงรูปแบบการทำงานของ If..Clause

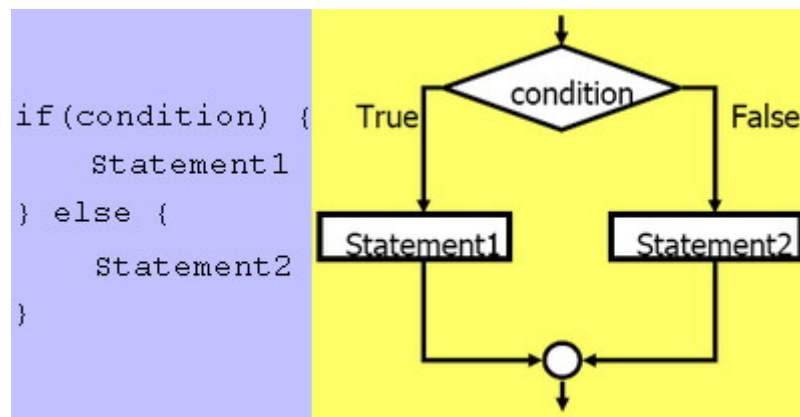
โดยปกติแล้วโปรแกรมที่เขียนจะมีลำดับการทำงานจากบนไปล่าง ทีละคำสั่ง จบหนึ่งคำสั่งถึงไปทำอีกคำสั่งหนึ่ง แต่ถ้าต้องการให้มันทำงานเป็นเงื่อนไข ก็ต้องตัดสินใจว่าจะทำคำสั่งหนึ่งหรือไม่ โดยขึ้นกับอีกคำสั่งหนึ่ง จะต้องใช้ control statement จำพวก if..else

โดยวงเล็บปิดจะเป็นตัวบอกว่าจบ condition แล้ว และเริ่ม statement ที่จะต้องทำงานเมื่อถูกเงื่อนไข และถ้าเราต้องการให้มีการ execute statement หลาย ๆ statement ในเงื่อนไขเดียว เราต้องใช้เครื่องหมาย '{' และ '}' (วงเล็บปีกกา) ครอบกลุ่มคำสั่งเหล่านั้นไว้ เช่น

```

if ( ) {
// statement_1
// statement_2
// ...
// statement_n
}
  
```

ถึงแม้ว่า statement หลัง if จะมีแค่ statement เดียว เราก็ควรจะใส่วงเล็บปีกกาครอบไว้เสมอ เพื่อความเป็นระเบียบ



รูป 2.10 แสดงรูปแบบการทำงานของ If Statement

2) Switch Case

switch..case ก็เป็นการเขียน if clause แบบหนึ่งที่ยั่งยืน มีรูปแบบ syntax ดังนี้

```

switch(<expression>) {
    case <value> : <statement>
    case <value> : <statement>
    case <value> : <statement>
    .....
    [default : <statement>]
}
  
```

ที่ expression นั้นจะเป็นตัวแปร หรือ method ใด ๆ ก็ได้
โดย default นั้นเปรียบเสมือน else ... ซึ่งจะเป็น option ที่มีก็ได้ ไม่มีก็ได้

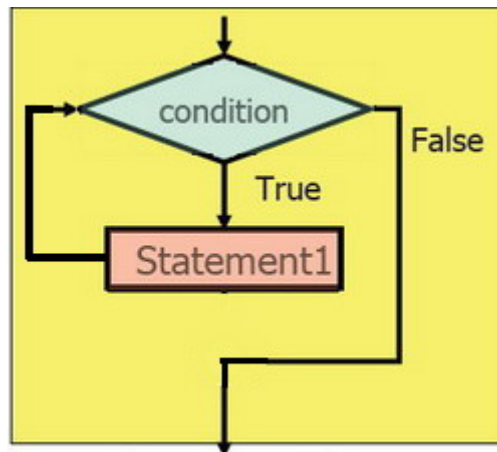
2.5.3. Loop Statement

การทำอะไรบางอย่างที่เป็นการทำซ้ำ ๆ กัน เราจะเรียกว่าการวนลูป คือการทำงานเป็นวงรอบนั่นเอง

โดย loop ในภาษานี้จะมีอยู่ 3 แบบคือ

- while()..
- do..while()
- for(; ;)

1) while loop

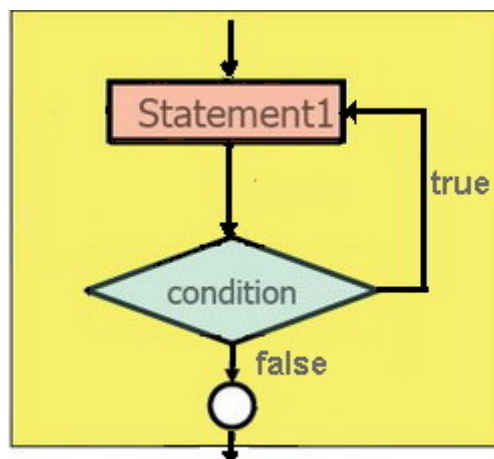


รูป 2.11 แสดงรูปแบบการทำงานของ While loop

จาก flow chart ข้างบน คือจะมีการเช็คเงื่อนไขก่อนเข้าลูป ถ้าเป็นจริง ก็
จะทำงาน แล้วก็จะกลับมาเช็คเงื่อนไขใหม่ไปเรื่อยๆ โดยมี syntax ดังนี้

```
while(cond)
    statement
```

2) do..while loop



รูป 2.12 แสดงรูปแบบการทำงานของ Do..While loop

มีลักษณะคล้ายๆกับ while loop แต่ต่างกันตรงที่ทำงาน statement ใน
loop ก่อน1ครั้งแล้วจึงค่อยเช็คconditionมี syntax ดังนี้

```
do
    statement
while(cond);
```

3) for..loop

for loop เป็นเหมือนกับ while loop ที่ลดรูปลงมา ไว้ใช้สำหรับการวนลูปที่กำหนดจำนวนครั้งเอาไว้ โดยมี รูปแบบ syntax ดังนี้

```
for(initial_expr; condition; update_expr)
    statement
```

2.6. MS SQL Server 2000

2.6.1. SQL Server 2000

SQL Server 2000 เป็นโปรแกรมจัดการฐานข้อมูลระดับเซิร์ฟเวอร์ที่มีขีดความสามารถในการรองรับข้อมูลขนาดใหญ่ที่ช่วยให้การบริหารจัดการฐานข้อมูลมีประสิทธิภาพ รวดเร็ว สนองตอบต่อความต้องการขององค์กรขนาดใหญ่

SQL Server 2000 เป็นระบบฐานข้อมูลและโซลูชันการวิเคราะห์ที่สมบูรณ์แบบ ซึ่งนำเสนอความน่าเชื่อถือและประสิทธิภาพ ด้านการขยายระบบที่เร็วและองค์กรธุรกิจต้องการ ด้วยการรองรับ XML และ HTTP ทำให้การเข้าถึงและการแลกเปลี่ยนข้อมูลนั้นทำได้ง่ายขึ้น ในขณะที่ความสามารถในการวิเคราะห์อันทรงพลังยังช่วยเพิ่มคุณค่าของข้อมูล และด้วยความพร้อมของระบบที่ดีขึ้น ทำให้ความสามารถในการทำงานอย่างต่อเนื่อง ของระบบนั้นสูงขึ้น, จัดการกับงานในแต่ละวันโดยอัตโนมัติได้ดีขึ้น รวมทั้งปรับปรุงเครื่องมือด้านการเขียนโปรแกรม และการพัฒนาความเร็วด้านการบริการได้ดีขึ้นเช่นกัน

SQL Server 2000 ประกอบไปด้วยเวอร์ชันต่าง ๆ ดังต่อไปนี้

- O Personal Edition** เป็นระบบจัดการฐานข้อมูลที่ใช้ได้เฉพาะเครื่องส่วนตัวเท่านั้น () ไม่สามารถใช้งานร่วมกับ SQL Server 2000 เวอร์ชันอื่นได้โดยตรง ส่วนใหญ่นิยมนำมาใช้ในการทดสอบการเขียนแอปพลิเคชัน ใช้ติดตั้งบน Windows 9x, ME, XP
- O Standard Edition** เป็นระบบจัดการฐานข้อมูลที่สามารถรองรับเซิร์ฟเวอร์ได้สูงสุดถึง 4 CPU และอ้างหน่วยความจำได้ถึง 2 GB นิยมนำมาใช้งานกับองค์กรขนาดใหญ่ ต้องติดตั้งบน Windows NT 4, 2000 หรือ 2003 Server แต่ถ้าติดตั้งบน Windows 9x, ME, XP จะติดตั้งให้เฉพาะส่วนที่เป็น Client Tools หรือ Connectivity เท่านั้น

O Enterprise Edition เป็นระบบจัดการฐานข้อมูลที่สามารถรองรับเซิร์ฟเวอร์ได้สูงสุดถึง 32 CPU และอ้างหน่วยความจำได้ถึง 64 GB นิยมนำมาใช้งานกับองค์กรที่มีขนาดใหญ่และมีสาขามากมาย เพราะมีระบบสนับสนุนการทำงานมากมาย เช่น การทำ Data Mining, Data Warehouse เป็นต้น ต้องติดตั้งบน Windows NT 4, 2000 หรือ 2003 Server เท่านั้น

2.6.2. TSQL (Transact-SQL)

เป็นคำสั่งที่ใช้ในการคิวรี (Query) ข้อมูลที่ต้องการ ประกอบไปด้วยคำสั่ง 3 ประเภท คือ

1. **Data Definition Language (DDL)** เป็นคำสั่งที่ใช้ในการจัดการอ็อบเจกต์ต่าง ๆ ของฐานข้อมูล เช่น ฐานข้อมูล ตาราง วิว ได้แก่คำสั่ง CREATE, ALTER, DROP
2. **Data Manipulation Language (DML)** เป็นคำสั่งที่ใช้ในการจัดการกับข้อมูลที่อยู่ในฐานข้อมูล ได้แก่คำสั่ง SELECT, INSERT, UPDATE และ DELETE
3. **Data Control Language (DCL)** เป็นคำสั่งที่ใช้ในการควบคุมการทำงานของเครื่อง เช่นการกำหนดสิทธิให้กับผู้ใช้งาน ได้แก่คำสั่ง GRANT เป็นต้น

SQL Server 2000 ได้ยึดมาตรฐาน ANSI SQL-92 ในการกำหนดคำสั่งที่ใช้กัน แต่ก็มีเพิ่มเติมขีดความสามารถในการทำงานให้ดียิ่งขึ้น นอกเหนือไปจากคำสั่งมาตรฐาน ทำให้มีขีดความสามารถสูง มีความยืดหยุ่นสูง โดยเรียกคำสั่ง SQL นี้ว่า Transact-SQL หรือ T-SQL

การเขียนคำสั่ง SELECT

ใช้สำหรับเลือกข้อมูลในตาราง มีรูปแบบการเขียนดังนี้

```
SELECT [Predicate] { * | [Table.]Field1 [AS Alias1]
[, [Table.]Field2 [AS Alias2] [, ...]]}
FROM Table[, ...]
[WHERE Search-Criteria]
[GROUP BY Group-List]
[HAVING Search-Criteria]
[ORDER BY Sort-Criteria]
```

SELECT เป็นคำสั่งที่ใช้กำหนดรายชื่อ Attribute (Field) ของ Relation (Table) ที่ต้องการให้แสดง

FROM เป็นคำสั่งที่ใช้สำหรับกำหนดรายชื่อ Relation (Table) ที่เป็นแหล่งข้อมูลที่ต้องการนำมาแสดงผล

WHERE เป็นคำสั่งที่ใช้สำหรับกำหนดเงื่อนไขในการเลือกข้อมูลจาก Relation (Table) ที่กำหนดจากส่วนของ FROM ขึ้นมาแสดง คำสั่งในส่วนนี้จะกำหนดหรือไม่ก็ได้ แต่ถ้าไม่กำหนดจะถือว่าต้องการทำงานกับทุก Tuple (Record) ใน Relation (Table) นั้น การกำหนดเงื่อนไขนั้นประกอบไปด้วยโอเปอเรเตอร์ Comparison, BETWEEN..AND, IN, LIKE, IS NULL, AND, OR, DISTINCT, TOP, ASC, DESC, GROUP BY, HAVING, ALL, COMPUTE, COMPUTE BY, ROLLUP, CUBE

วิธีการเขียนภาษา SQL

การเขียนภาษา SQL มีข้อกำหนดดังนี้

1. เขียนได้ตั้งแต่ 1 บรรทัดเป็นต้นไป
2. ส่วนต่าง ๆ ของคำสั่งสามารถเขียนแยกบรรทัดได้
3. ภายใน 1 อนุประโยคไม่สามารถแยกบรรทัดได้
4. คำสั่ง SQL จะเรียกใช้ที่ SQL Prompt (SQL>)
5. ประโยคคำสั่งสามารถทำงานได้หลาย ๆ ครั้งในขณะที่ยังอยู่ในบัฟเฟอร์
6. ทุก ๆ ประโยคคำสั่งต้องจบด้วยเครื่องหมายเซมิโคลอน (;)

ตัวอย่างของการเขียนภาษา SQL มีดังนี้

แบบที่ 1

เมื่อ * หมายความว่า ข้อมูลจากทุกฟิลด์ที่ต้องการนำมาแสดงผล

```
SELECT * FROM LAWS;
```

แบบที่ 2

```
SELECT
*
FROM
LAWS;
```

แบบที่ 3

```
SELECT *  
FROM LAWS;
```

ไม่ว่าจะเขียนภาษา SQL แบบใดก็ตาม (แบบที่ 1 – แบบที่ 3) สามารถทำงานได้ ผลลัพธ์ออกมาเป็นเช่นเดียวกันทั้งหมด

สังเกตว่าทุกประโยคคำสั่ง SQL มาตรฐานจะสิ้นสุดลงด้วยเครื่องหมายเซมิโคลอน (;) แต่สำหรับ SQL Server จะไม่มีการใช้เครื่องหมายดังกล่าว ดังนั้นเวลาใช้งานต้องระมัดระวังด้วย เพราะถ้าใส่ลงไปจะมี Error เวลาสั่ง Run

คุณสมบัติอื่นของภาษา SQL

เพื่อให้ภาษา SQL มีคุณสมบัติในการทำงานเพียงพอต่อการนำไปใช้งาน จึงมีการกำหนดคุณสมบัติต่าง ๆ ของภาษา SQL เพิ่มเติมขึ้นมา ดังนี้

1. การเพิ่มนิพจน์ (Expression) ขึ้นมาเพื่อใช้เป็นเงื่อนไขในการค้นหา
2. การสร้างชื่อคอลัมน์ใหม่ขึ้นมา (Alias)
3. การเชื่อมต่อกอลัมน์ (Concatenation Operator)
4. การแสดงค่าเฉพาะ (Literals)
5. การทำงานกับค่าที่เป็นค่าว่าง (Null)
6. การเรียกดูข้อมูลที่มีค่าไม่ซ้ำกัน (Distinct)
7. การเรียงลำดับข้อมูล (Order By)
8. การเรียกดูข้อมูลโดยมีเงื่อนไข (Where Clause)

JOIN

เป็นการนำตารางหลาย ๆ ตารางมาคิวรีพร้อมกัน เพื่อหาข้อมูลที่ต้องการ

Inner Join เป็นการนำตาราง 2 ตารางมา join กัน ตามเงื่อนไขที่กำหนด

- Equi JOIN จะมีการแสดงข้อมูลซ้ำกันในคอลัมน์ที่นำมาเปรียบเทียบกัน
- Natural JOIN จะไม่นำข้อมูลคอลัมน์ที่เปรียบเทียบมาแสดง

Cross Join (Unrestricted JOIN) เป็นการ JOIN ที่แสดงถึงความเป็นไปได้ของผลลัพธ์ที่สามารถเกิดขึ้นได้ (Cartesian Product)

Outer Join การ JOIN ที่อยู่นอกเหนือเงื่อนไขที่กำหนด ประกอบไปด้วย

- LEFT OUTER JOIN (*) ยึดรายการจากตารางหลักเป็นหลัก

- RIGHT OUTER JOIN (= *) ขีดรายการจากตารางที่สองเป็นหลัก
Outer Join เป็นการ JOIN ตารางเดียวกัน แต่ตั้งชื่อ alias ต่างกัน

รูปแบบความสัมพันธ์

เนื่องจากตารางความสัมพันธ์กำหนดให้แต่ละตารางต้องมีความสัมพันธ์กัน ซึ่งเราสามารถแยกความสัมพันธ์ของตารางออกเป็น 4 แบบ ดังนี้

1. **หนึ่งรายการต่อหนึ่งรายการ (One to One)** ข้อมูลรายการหนึ่งไปสัมพันธ์กับข้อมูลของอีกตารางหนึ่งได้เพียงรายการเดียวเท่านั้น เช่น คนหนึ่งคนมีจังหวัดเกิดได้เพียงจังหวัดเดียวเท่านั้น
2. **หนึ่งรายการต่อหลายรายการ (One to Many)** ข้อมูลรายการหนึ่งไปสัมพันธ์กับข้อมูลของอีกตารางหนึ่งได้หลายรายการ เช่น ตำแหน่งพนักงานขายสามารถมีได้หลายคน
3. **หลายรายการต่อหนึ่งรายการ (Many to One)** ข้อมูลหลายรายการไปสัมพันธ์กับข้อมูลของอีกตารางหนึ่งได้เพียงรายการเดียวเท่านั้น เช่น พนักงานขายหลายคนทำงานอยู่บริษัทเดียวกัน
4. **หลายรายการต่อหลายรายการ (Many to Many)** ข้อมูลหลายรายการไปสัมพันธ์กับข้อมูลของอีกตารางหนึ่งได้หลายรายการ เช่น พนักงานคนหนึ่งสามารถรับปริญญาตรีจากมหาวิทยาลัยได้หลายใบ ในขณะที่เดียวกันมหาวิทยาลัยก็สามารถผลิตบัณฑิตได้หลายคน

ในทางปฏิบัติพบว่าการออกแบบฐานข้อมูลที่ดี จะมีความสัมพันธ์ของตารางเป็นแบบหนึ่งรายการต่อหนึ่งรายการ หรือแบบหนึ่งรายการต่อหลายรายการ และไม่ค่อยพบเห็นตารางที่มีความสัมพันธ์แบบหลายรายการต่อหลายรายการโดยตรง แต่จะใช้วิธีการสร้างตารางความสัมพันธ์แบบหนึ่งรายการต่อหนึ่งรายการหรือแบบหนึ่งรายการต่อหลายรายการเข้ามาทำหน้าที่เชื่อมความสัมพันธ์แทน

การใช้ Sub Query

ในบางครั้งผลลัพธ์ที่ได้จากการ JOIN สามารถใช้ Subqueries แทนได้เช่นกัน แต่ก็มีบ้างที่เพียง Subqueries เท่านั้นจึงจะสามารถหาผลลัพธ์ที่ต้องการได้ แต่หากพูดถึงประสิทธิภาพในการทำงานของทั้งสองก็ไม่แตกต่างกันมากนัก โดยทั้งนี้ขึ้นอยู่กับเงื่อนไขในการระบุอีกด้วย

ตามปกติในการใช้งาน Subqueries จะถูกรอไว้ด้วยวงเล็บเพื่อให้เห็นถึงส่วนของคิวรีย่อยไว้อย่างชัดเจน โดยที่คิวรีย่อยสามารถมีคิวรีย่อยได้อีกต่อไปเรื่อย ๆ หลายชั้นไม่จำกัด

Union

เป็นตัวเชื่อมผลลัพธ์ที่เกิดจากต่างคิวรีกัน แต่มีลักษณะการแสดงผลตามลำดับของคอลัมน์คล้าย ๆ กันแล้วนำข้อมูลเหล่านี้มาจัดแสดงได้พร้อมกัน

VIEW

คือตารางเสมือนที่เกิดจากการจำลองตารางข้อมูล โดยนำเฉพาะบางคอลัมน์ บางแถว หรือแม้แต่สามารถนำข้อมูลบางส่วนจาก 2 ตารางขึ้นไป มาแสดงไว้ภายในตารางที่วันนี้ได้ เช่น

ลำดับ ที่	ชื่อ-สกุล	โทรศัพท์	E-Mail	
1	นายจิตเกษม พัฒนาศิริ	014241742	chitpatt@oalweb.com	} ตารางหลัก
2	นางสาวจิตติมา มโนหมั่นศรีตรา	040100464	thitima@oalweb.com	
3	นายสมชาย พงษ์เกษม	029885025	somchaip@oalweb.com	

ตาราง 2.10 แสดงผล ที่ได้จาก Query

ชื่อ-สกุล	E-Mail	
นายจิตเกษม พัฒนาศิริ	chitpatt@oalweb.com	} วิว
นางสาวจิตติมา มโนหมั่นศรีตรา	thitima@oalweb.com	

ตาราง 2.11 แสดงวิว ที่ได้จาก Query

จะเห็นได้ว่าการสร้างวิว คือ การนำคิวรีมาสร้างให้เป็นออบเจกต์หนึ่งของฐานข้อมูล มีประโยชน์กับคิวรีที่มีการใช้งานบ่อย ๆ ทำให้สามารถแสดงข้อมูลได้อย่างรวดเร็ว

SELECT INTO

ใช้สำหรับการทำสำเนาข้อมูลจากตารางหนึ่งมาเป็นอีกตารางหนึ่ง มีรูปแบบการเขียนดังนี้

```
SELECT Column_list (Column1,Column 2, ...)
INTO new_TableName
FROM Table_list (TableName 1, TableName 2, ...)
```

INSERT

ใช้สำหรับเพิ่มแถวข้อมูลลงในตาราง มีรูปแบบการเขียนดังนี้

```
INSERT INTO TableName (Field1, Field2, ...) VALUES (Value1, Value2, ...)
INSERT INTO TableName SELECT-STATEMENT
```

DELETE

ใช้สำหรับลบแถวข้อมูลในตาราง มีรูปแบบการเขียนดังนี้

```
DELETE FROM TableName WHERE Search-Criteria
```

UPDATE

ใช้สำหรับแก้ไขข้อมูลในตาราง มีรูปแบบการเขียนดังนี้

```
UPDATE TableName SET ColumnName = Value, ... [WHERE Search-
Criteria]
```

ในทางปฏิบัติจะไม่ค่อยเห็นการใช้งาน Transact-SQL โดยตรงมากนัก ทั้งนี้เพราะส่วนใหญ่นำคำสั่ง SQL ไปฝังไว้กับ Application ที่พัฒนาขึ้นมาจาก VB, Delphi, PB เป็นต้น

2.6.3. Stored Procedure คืออะไร

Stored procedures เป็นกลุ่มคำสั่ง SQL ร่วมกับคำสั่งที่เป็นพวก control flow พื้นฐานอย่างพวก if then else, for loop , while loop เป็นต้น Stored procedures จะแตกต่างกับคำสั่ง SQL หรือกลุ่มคำสั่ง SQL ตรงที่มันมีการทำ pre-compiled ไว้ก่อนล่วงหน้า คือทุกครั้งที่เราสร้าง store procedure ในครั้งแรก ตัว SQL Server จะคอมไพล์และเก็บคำสั่งนั้นไว้ในตารางระบบ (system table) ของตัว SQL Server แล้วจึงนำคำสั่งนั้นไป execute หลังคอมไพล์

คำสั่งที่อยู่ใน Stored procedures ก็จะคล้ายๆกับคำสั่งที่เราเห็นทั่วไปในภาษาโปรแกรม แต่ syntax ของมันค่อนข้างจะผสมระหว่างภาษา C กับ Pascal ซึ่งมันสามารถรับและรีเทิร์นค่าพารามิเตอร์เหมือนกับฟังก์ชันทั่วๆไปได้ ซึ่งผลลัพธ์ของการเรียกใช้ Store Procedure มีความเป็นไปได้ดังนี้

1. ไม่ส่งอะไรกลับมาเลย กรณีนี้คำสั่งใน Store Procedure จะสั่งให้ update ,addnew หรือ delete หรือทำอะไรก็ตามที่ไม่ต้องการส่งค่ากลับมา
2. ส่งค่ากลับมาเป็น recordset
3. ส่งค่ากลับเป็นตัวแปรเช่นตัวเลขหรือตัวอักษร

ข้อดีของ store procedure

Stored procedure มีคุณสมบัติดีกว่าการคิวรี (query) ข้อมูลธรรมดา เพราะว่าการคิวรี ข้อมูลธรรมดาจากหน้า ASP, ASP.NET , PHP หรือ script อื่นๆ หรือจากวินโดวส์ แอปพลิเคชันนั้น ทุกครั้งที่มีการส่งคำสั่ง sql ไปที่เครื่องที่ทำหน้าที่เป็น Database Server มันจะทำการคอมไพล์คำสั่ง query นั้นใหม่ทุกครั้งที่มีการเรียก นั่นหมายถึงถ้าเรียกบ่อยครั้งก็คอมไพล์คำสั่ง query ร้อยครั้งแล้วค่อยนำคำสั่ง query นั้นไป execute อีกที

แนวคิดของ Store Procedure จึงถูกนำมาแก้ไขปัญหาดังกล่าว โดยการเขียนชุดคำสั่ง query รวบรวมเอาไว้บน Database Server แล้วคอมไพล์ชุดคำสั่งเหล่านั้นไว้ครั้งแรกครั้งเดียว เมื่อเราต้องการ query ข้อมูล ก็สั่งให้ Store Procedure ที่เราต้องการให้ทำงานแทนการส่งชุดคำสั่ง sql ไปคอมไพล์และ execute ใหม่ นอกจากนี้ข้อดีอีกอย่าง Store Procedure คือเราสามารถส่งพารามิเตอร์ทั้งเข้าและออก store procedure เพื่อมองให้เหมือนกับฟังก์ชันทั่วไปที่เราเขียน จะต่างกันก็ในด้านของความหมายเพราะ store procedure จะเป็นฟังก์ชันการทำงานทางด้านฐานข้อมูล

ลักษณะที่คล้ายกับ Programming Language

Stored procedures ของ SQL Server จะมีคุณสมบัติที่คล้ายคลึงกับภาษาโปรแกรมทั่วไปไปแยกเป็นข้อๆ ได้ดังนี้

- พารามิเตอร์ที่ส่งเข้าไปหรือรีเทิร์นกลับมามี data type คล้ายกับภาษาทั่วๆไปเช่น int , char ,nvarchar เป็นต้น
- มีชุดคำสั่งที่ทำงานเกี่ยวกับ database และคำสั่งพื้นฐานอย่างพวก if then else, loop ต่างๆ รวมถึงคำสั่งที่เรียกเอา store อื่นมาทำงานด้วย
- มีการส่งค่า status กลับในกรณีที่เรียก procedure อื่นซึ่งระบุว่าทำงานสำเร็จหรือไม่สำเร็จ
- มีการ Precompiled คือคอมไพล์ครั้งแรกแล้วเก็บไบนารี code ที่คอมไพล์ไว้เพื่อไม่ต้องคอมไพล์อีก
- ลดปัญหาการจราจบนเน็ตเวิร์กเพื่อไม่ต้องส่งคำสั่ง query ยาวๆไปหาเครื่องที่เป็น SQL Server เพราะจะส่งเพียงชื่อและพารามิเตอร์ไปเท่านั้น
- เพิ่มประสิทธิภาพในการเรียกใช้โค้ดที่ทำงานซ้ำๆกัน เพื่อจะได้ไม่ต้องเขียนโค้ดในหลายๆที่
- กำหนดสิทธิได้ว่าจะให้ user คนไหนมีสิทธิรัน Store Procedure ไหนบ้าง

- ความเร็วในการคิวรี่ที่รวดเร็วมก เพราะในตอน precompile ถ้าโค้ดที่เป็น Transact-SQL ของเรามีความสลับซับซ้อนตัวคอมไพเลอร์จะทำการ Optimize โค้ดเหล่านั้นให้เราด้วย
- สามารถกำหนดได้ว่าจะให้ execute Store Procedure บนเครื่อง local หรือบนเครื่อง SQL ซึ่งมันจะช่วยให้เรื่องของการประมวลผลข้าม server ในกรณีที่เป็น Farm Server

รูปแบบของ Store Procedure

สามารถกำหนดชื่อของ Store Procedure ได้ความยาวสูงสุด 30 ตัวอักษรด้วยการกำหนด syntax ของ Store Procedure ดังนี้

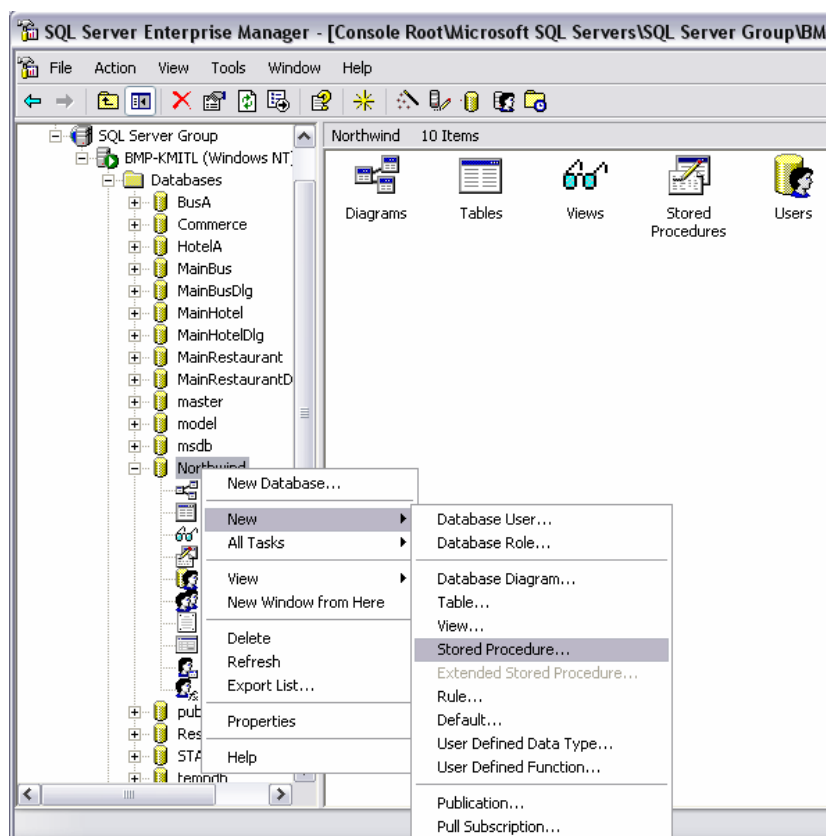
```
CREATE PROCEDURE [owner,] procedure_name [;number]
[@parameter_name datatype [=default] [OUTput]
...
[@parameter_name datatype [=default] [OUTput]
[FOR REPLICATION] | [WITH RECOMPILE] , ENCRYPTION
AS sql_statements
```

ภายหลังที่สร้าง Store Procedure เรียบร้อยแล้ว สามารถทดสอบ Store Procedure โดยผ่านทางกรเขียนโค้ดที่ใช้ ADO หรือผ่านทางโปรแกรม Query Analyzer ที่มีมากับ SQL Server ได้

เริ่มทดลองสร้าง Store Procedure

- 1) ให้เปิด Enterprise Manager ของ SQL-Server ขึ้นมา
- 2) เลือกไปที่ database NorthWind
- 3) Click ขวาที่ Store Procedure ดังรูป
- 4) เลือก New Store Procedure
- 5) ป้อนโค้ดต่อไปนี้ลงไปบนหน้าต่างที่โผล่ขึ้นมา

```
create procedure all_employees
as select * from employees
GO
```

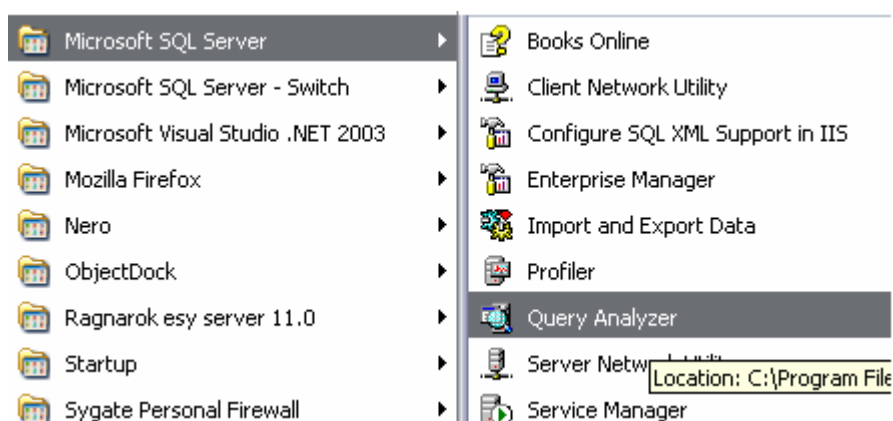


รูป 2.13 แสดงการสร้าง Stored Procedure

6) คลิก OK

การเรียกใช้ Store Procedure ผ่าน Query Analyzer

1) เปิดโปรแกรม Query Analyzer



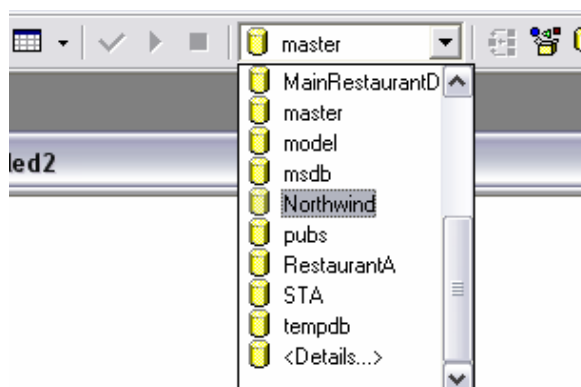
รูป 2.14 แสดงการเปิดโปรแกรม Query Analyzer

- 2) เลือก SQL Server และป้อน username password



รูป 2.15 แสดงการเปิดเลือกเปิด SQL Server

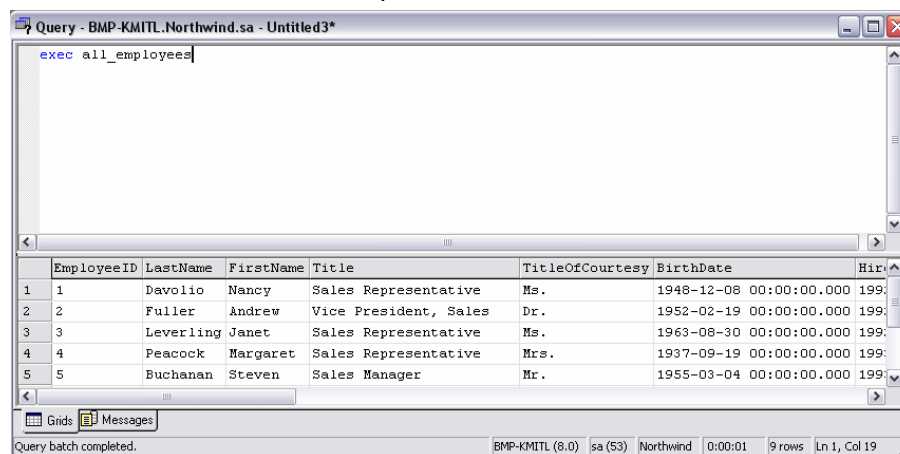
- 3) เลือก Database ไปที่ NorthWind



รูป 2.16 แสดงการเปิดเลือกเปิด Database NorthWind ใน SQL Server

- 4) พิมพ์คำสั่งต่อไปนี้

exec all_employees แล้ว click ปุ่ม  จะได้ผลลัพธ์เป็น recordset คืนกลับมา



รูป 2.17 แสดงผลจากการ Query โดยใช้ Store Procedure